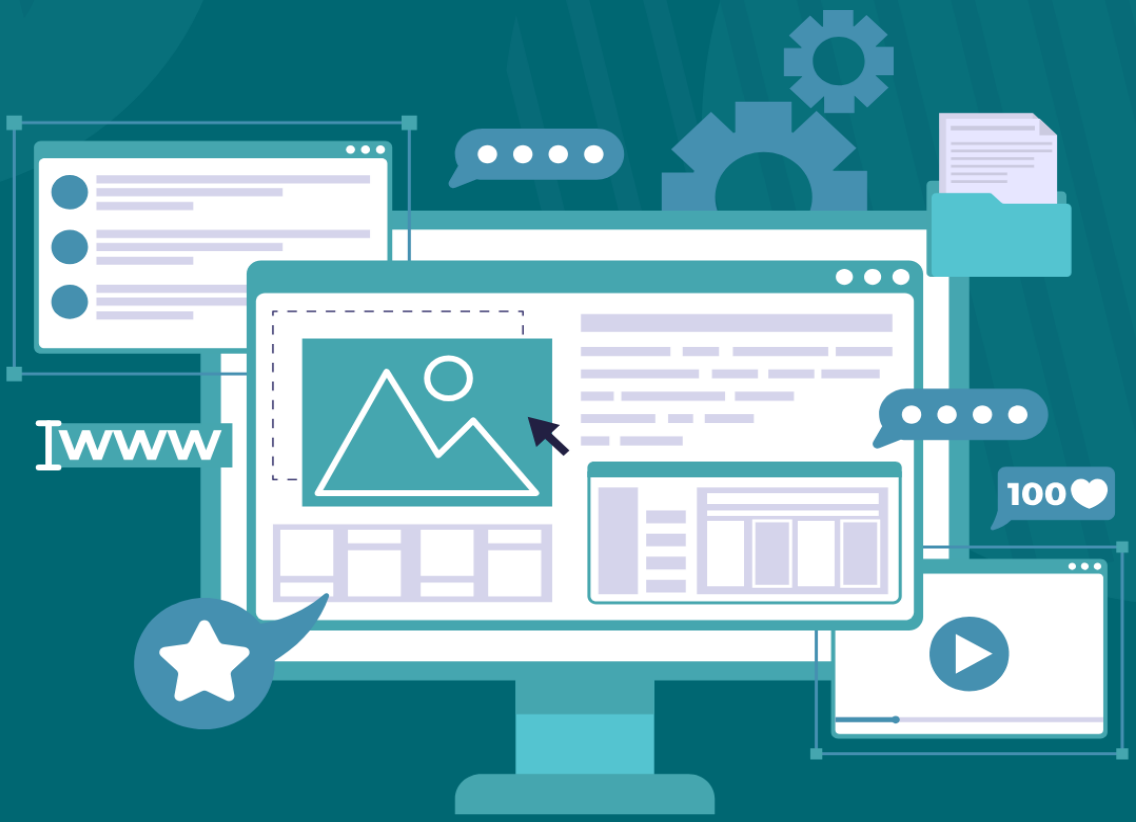
GCWUS




# WEB DESIGN & DEVELOPMENT

## LAB MANUAL 14

**Submitted By :**

Manahil Abid 10
Eman Nisar Ahmad 43

**Submitted To :**

Ma'am Shaista
Computer Science

**27-02-2026**

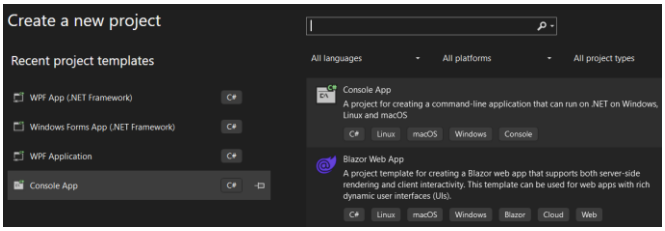
# *Form Creation*

## Task:

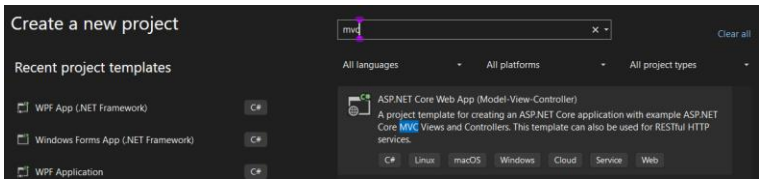
```
Objective:
Develop a simple ASP.NET Core MVC application using Entity Framework Core (Code First approach) to store Student/User registration data into SQL Server.

The application must allow a user to submit a form containing:
- Name
- Email
- Password
- Gender
- Age

After successful submission, data should be saved in the database and a success alert message should be shown.

Step 1: Install Required Packages
- Create a new ASP.NET Core MVC project in Visual Studio.
- Install NuGet packages:
  - Microsoft.EntityFrameworkCore
  - Microsoft.EntityFrameworkCore.SqlServer
  - Microsoft.EntityFrameworkCore.Tools

Step 2: Create Model Class
- Create a Models folder.
- Create class Student with properties: Id, Name, Email, Password, Gender, Age.
- Apply data annotations (Required, etc.)

Step 3: Create DbContext Class
- Create class StudentDBContext inheriting from DbContext.
- Accept DbContextOptions in constructor.
- Add DbSet<Student> Students { get; set; }

Step 4: Add Connection String in appsettings.json
- Add connection string with key DefaultConnection
- "ConnectionStrings": {
    "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True}

Step 5: Register Connection String in Program.cs
- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

Step 6: Migration & Update Database
- Use Package Manager Console or CLI:
  - Add-Migration InitialCreate
  - Update-Database

Step 7: Controller Injection and Create User Form Page (Create Template)
- Controller with constructor injection of DbContext.
- Scaffold Create Razor page for Student.
- Fields: Name, Email, Password, Gender, Age.

Step 8: HTTP POST Action to Submit Data
- [HttpPost] action accepting Student model.
- Validate ModelState.
- Save data using DbContext.

Step 9: Show Success Alert Message
- After successful submission, show alert: "Data successfully submitted" using TempData or JavaScript.

Step 10: Design the User Form
```

- Search & open ***Visual Studio 2022***.

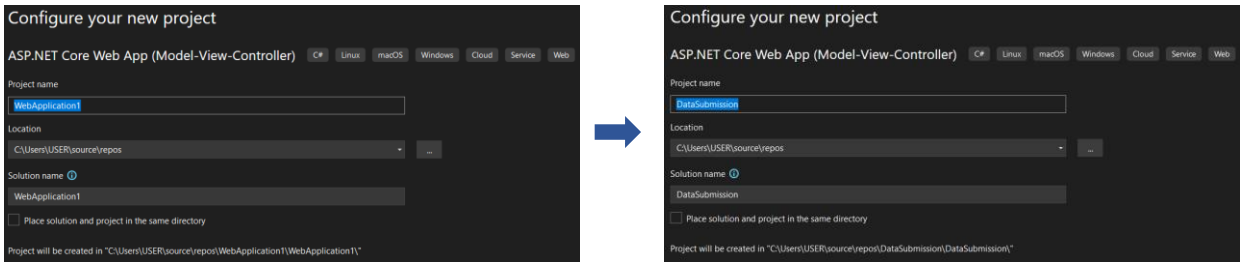

- Click on ***Create a new Project***:



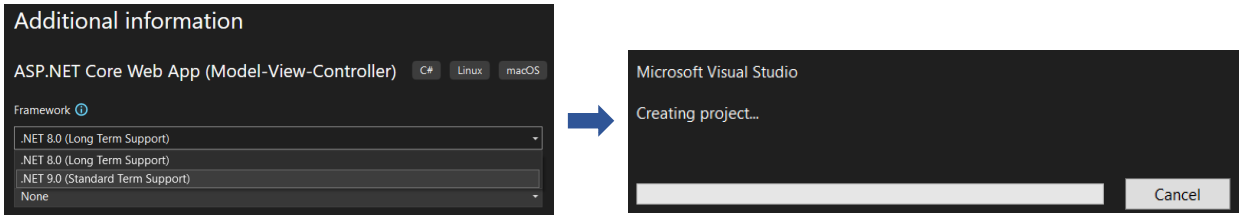
- Search ***MVC*** in the search bar:



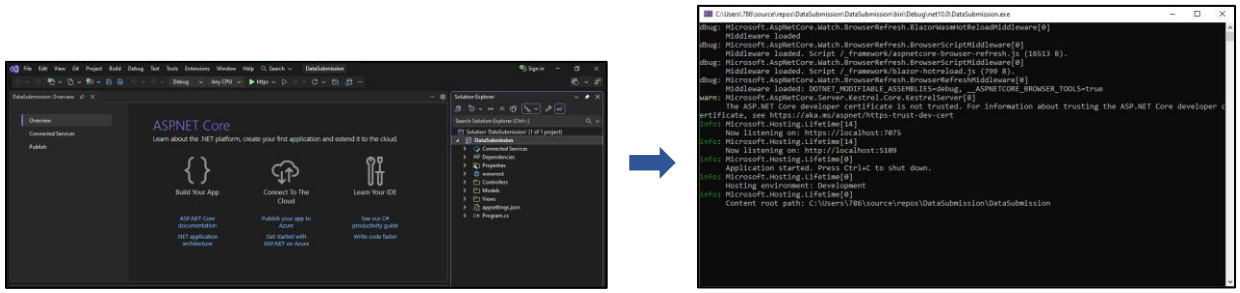

- Choose name for your project Like ***DataSubmission***:



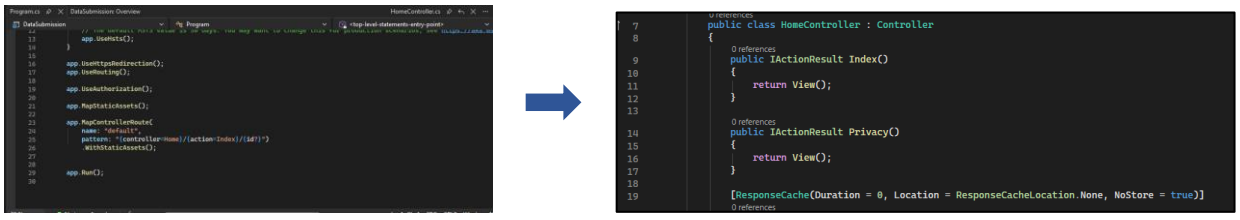

- Choose ***version*** & next:




- ASP.Net Core ***interface***:



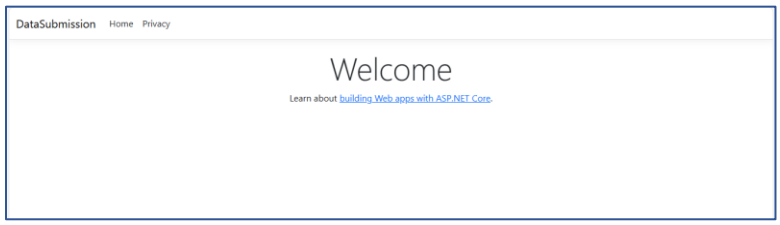

- Firstly ***program.cs*** file run then in it ***the Home controller index method*** will run and it return the index view which is actually the html page in home tab. The ***Action method*** runs.



- ***Model file*** will be html file which shows actually

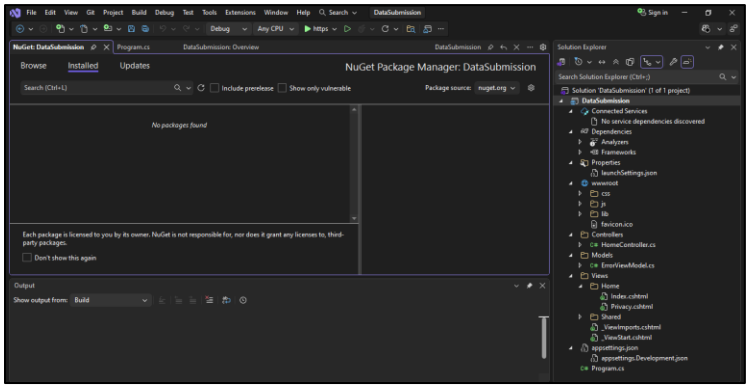
```
Program.cs          DataSubmission: Overview                    Index.cshtml
1   @{
2       ViewData["Title"] = "Home Page";
3   }
4
5   <div class="text-center">
6       <h1 class="display-4">Welcome</h1>
7       <p>Learn about <a href="https://learn.microsoft.com/aspnet/core">building Web apps with ASP.NET Core</a>.</p>
8   </div>
```

- ***Run*** the code this screen will appear.

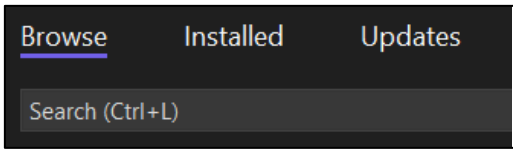


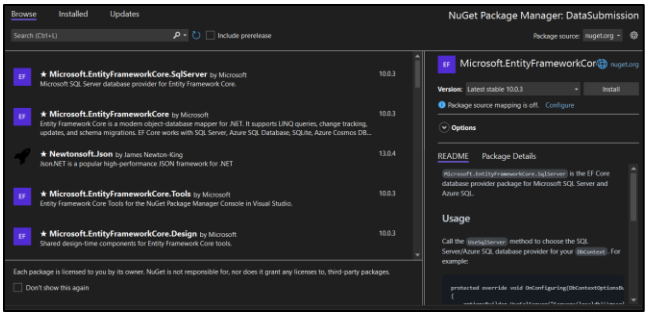

```
Step 1: Install Required Packages
- Create a new ASP.NET Core MVC project in Visual Studio.
- Install NuGet packages:
  - Microsoft.EntityFrameworkCore
  - Microsoft.EntityFrameworkCore.SqlServer
  - Microsoft.EntityFrameworkCore.Tools
```

***Install packages:*** Right click name of project "***DtataSubmission***"

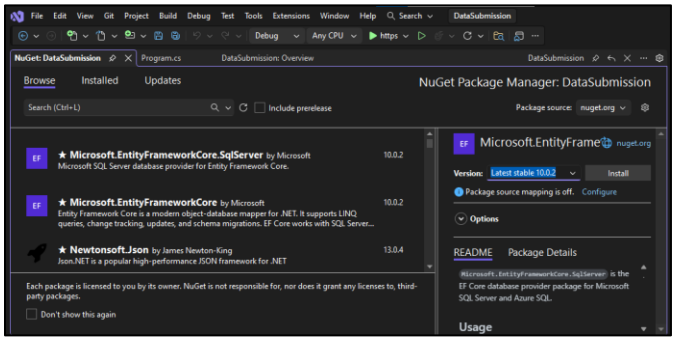



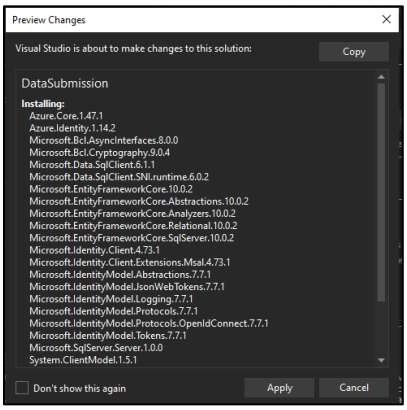

- Click on ***Browse***.



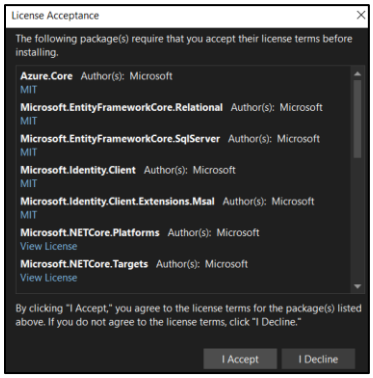



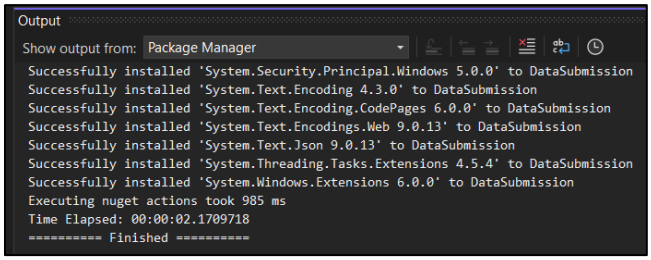

- Install first package ***Microsoft.EntityFrameworkCore.SqlServer*** by Microsoft & ***apply***.



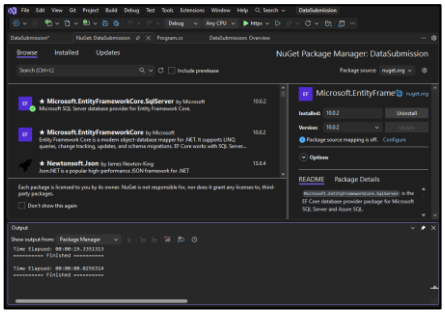



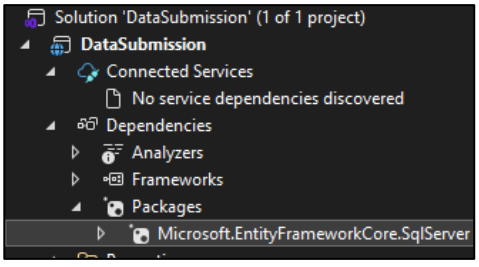

- Click ***I Accept*** then package successfully installed.



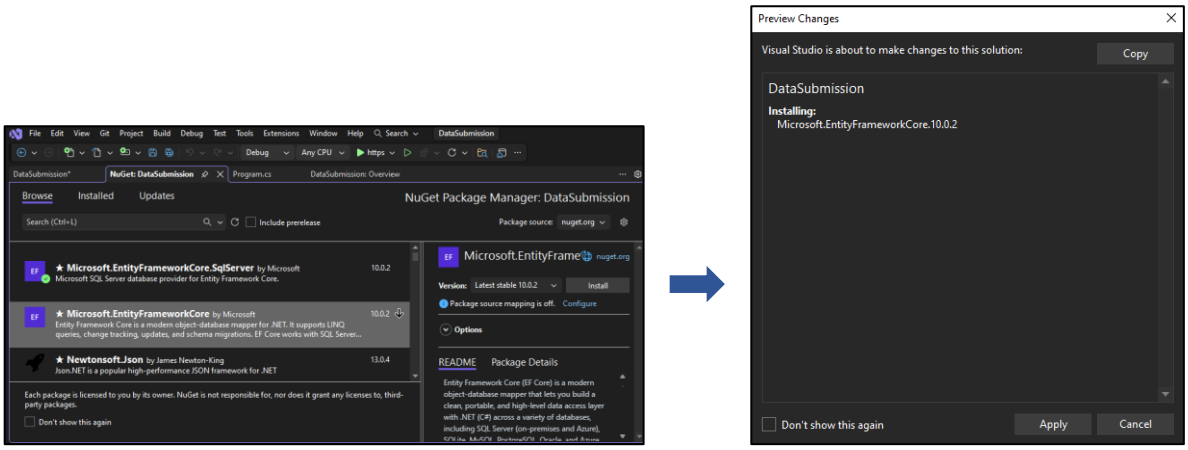




- The ***Green icon will appear on Package*** after installation Now check in ***Dependencies*** Package will be show if install completely.

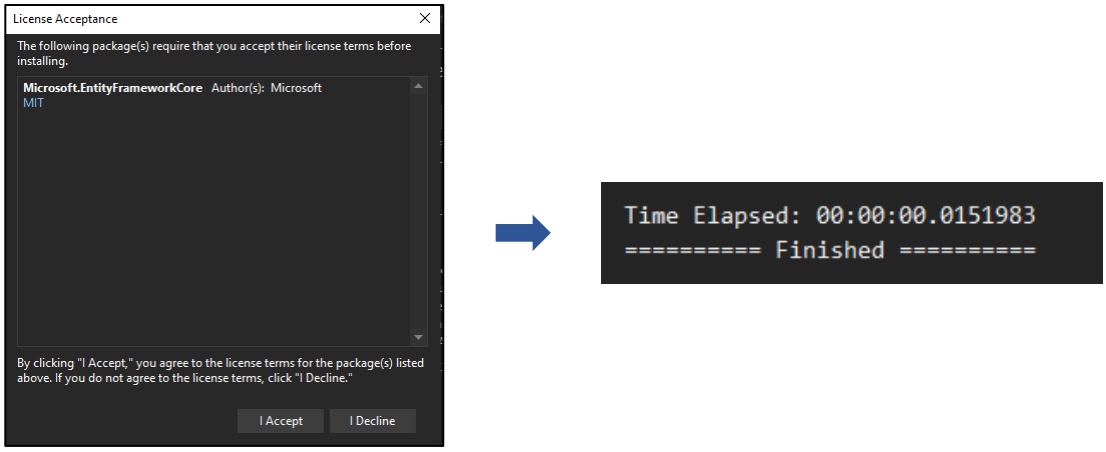




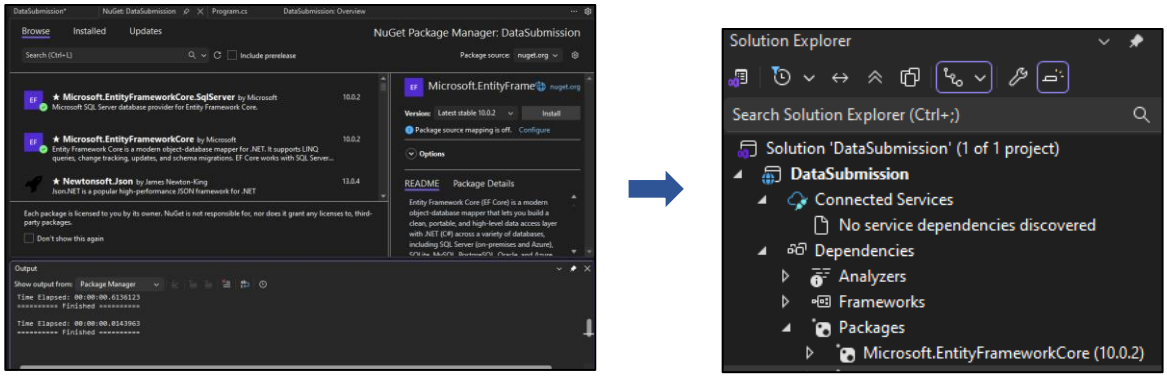
- Now Install ***second package*** & click ***Apply***:



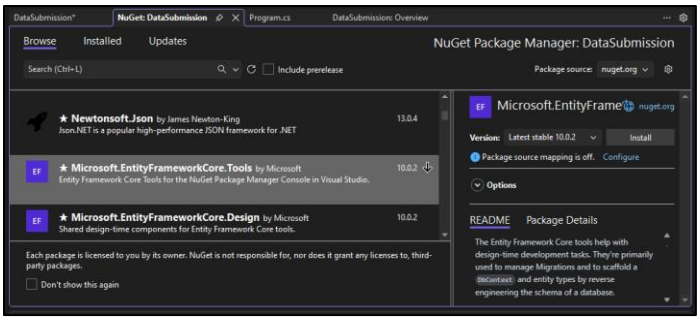

- Click ***I Accept*** then package successfully installed.

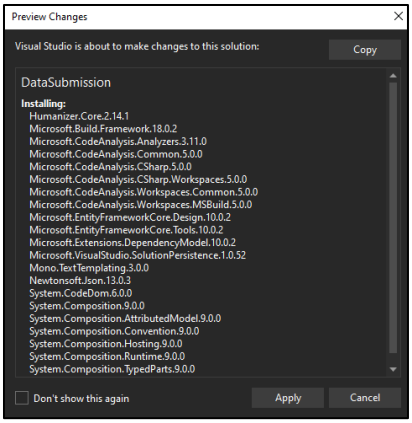



- The ***Green icon will appear on Package*** after installation Now check in ***Dependencies*** Package will be show if install completely.

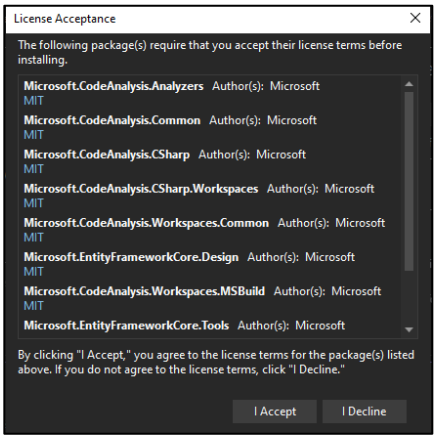


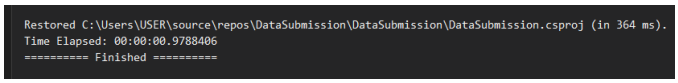

- Now Install ***third package*** & click ***Apply***:



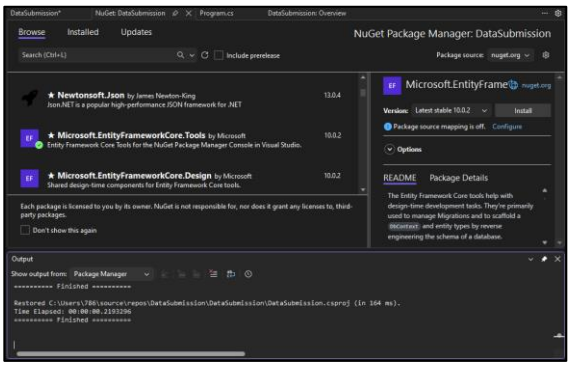



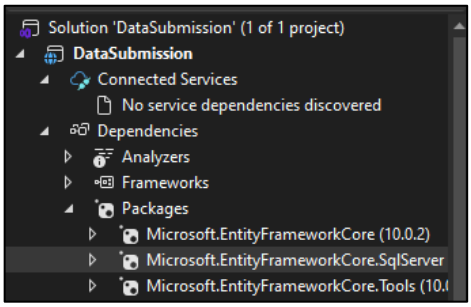

- Click ***I Accept*** then package successfully installed.



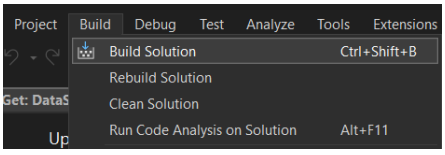



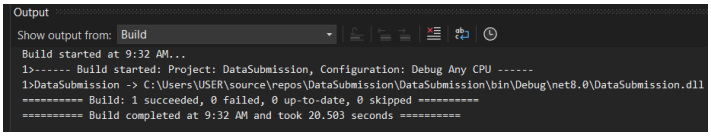

- The ***Green icon will appear on Package*** after installation Now check in ***Dependencies*** Package will be show if install completely.



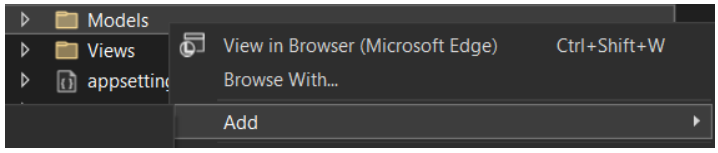

- After installing packages, In ***Build*** from Top, ***Build Solution*** no conflict occurs.



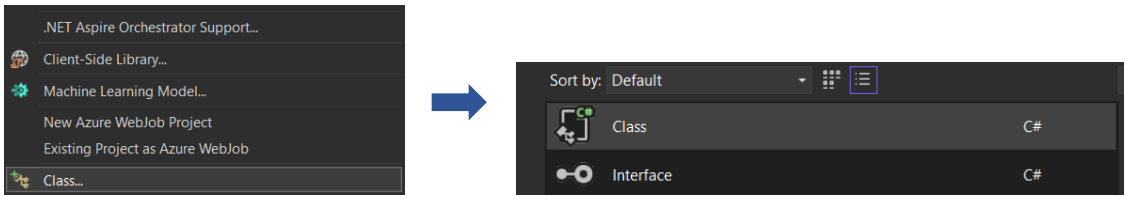



```
Step 2: Create Model Class
- Create a Models folder.
- Create class Student with properties: Id, Name, Email, Password, Gender, Age.
- Apply data annotations (Required, etc.)
```

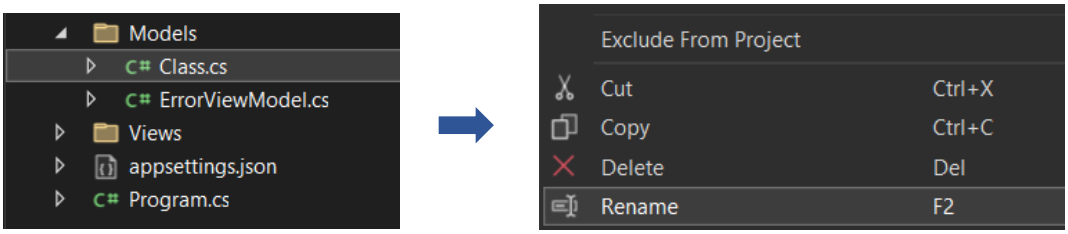
- ***Create Model class***: Model class is the one which is named as the table name we want create for example student
- We don't create database directly in SQL server instead we use code first approach.
- For creating model class go to model and right click it then go to ***ADD*** & then ***class***.



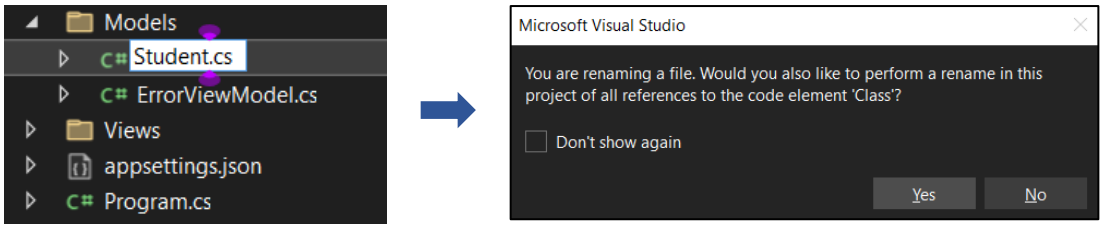

- After ***ADD*** choose class from list then click on class Class:



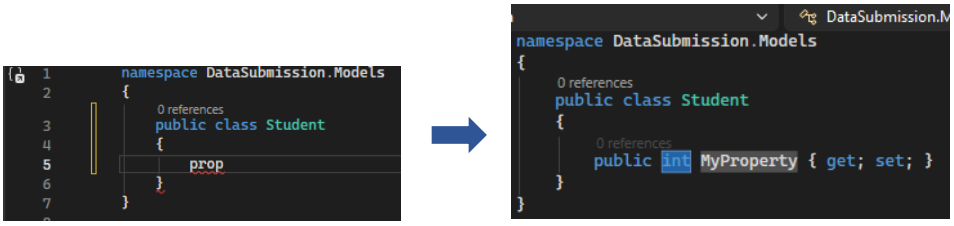

- ***Class.cs*** will create ***Now Right click*** & ***Rename***




- Change Name ***Student.cs*** & ***OK*** for message.




- ***Prop*** then click on ***tab***



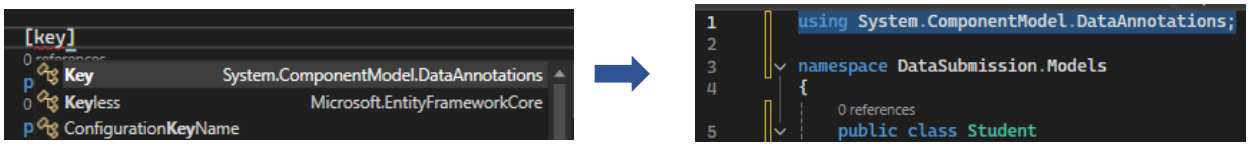
- First Id ***every table has id.***

```
namespace DataSubmission.Models
{
    0 references
    public class Student
    {
        0 references
        public int Id { get; set; }
    }
}
```

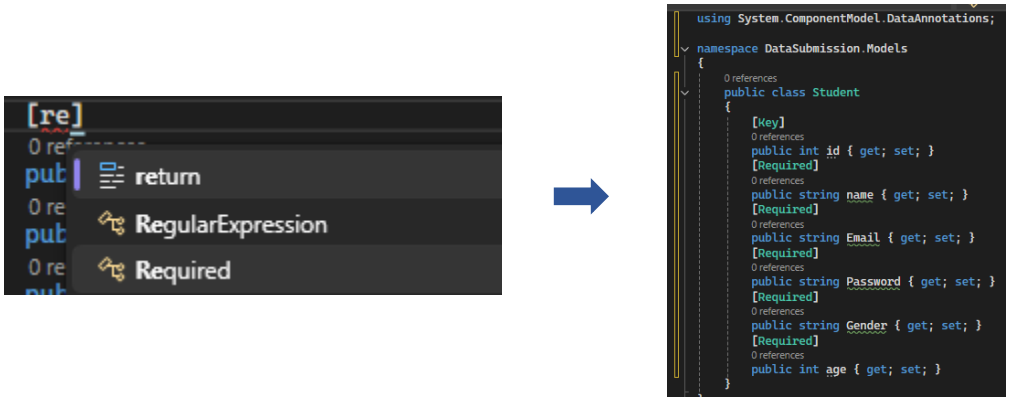
- ***Creating Form***: Id, name, gender & password.

```
namespace DataSubmission.Models
{
    0 references
    public class Student
    {
        0 references
        public int id { get; set; }
        0 references
        public string name { get; set; }
        0 references
        public string Email { get; set; }
        0 references
        public string Password { get; set; }
        0 references
        public string Gender { get; set; }
        0 references
        public int age { get; set; }
    }
}
```

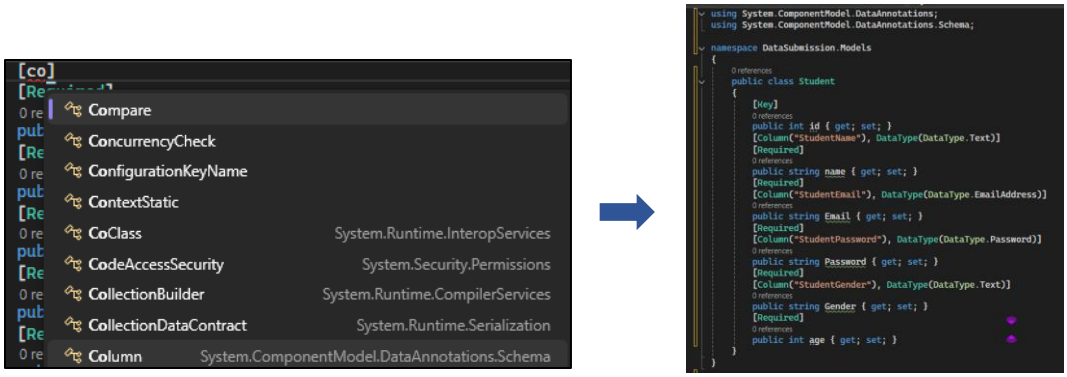
- ***Setting primary Key***: As we know each table has primary key in sql so make the id primary key. Key is in system.component.model. data annotations. Double click it now it show on top of code.




- ***Making required***: Making all things compulsory for it type [Required]

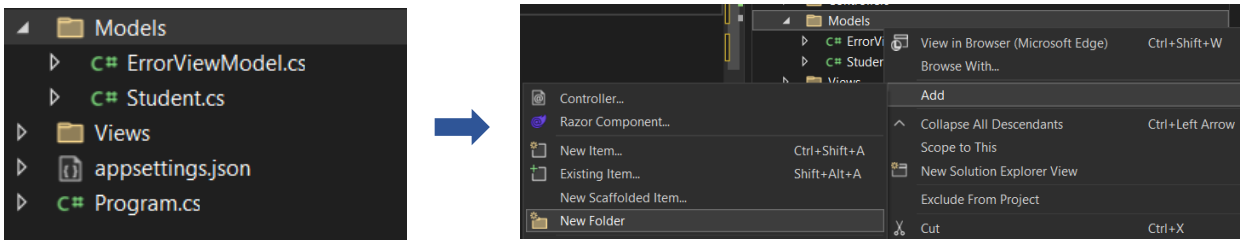



- ***Setting Column***: If we want that name is not shown in table (column name) instead student name show and so on then we use[column] and set it's name and datatype.

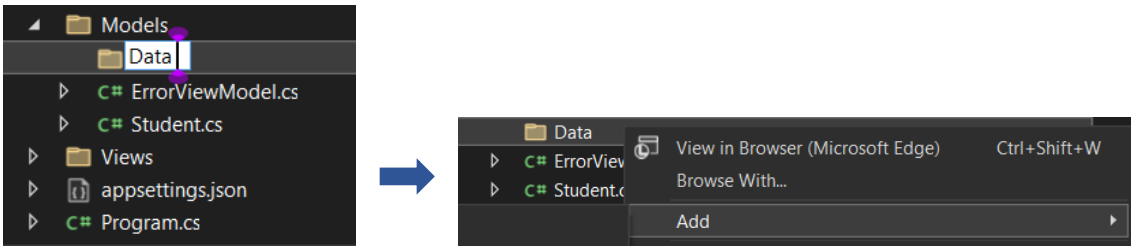



```
Step 3: Create DbContext Class
- Create class StudentDBContext inheriting from DbContext.
- Accept DbContextOptions in constructor.
- Add DbSet<Student> Students { get; set; }
```

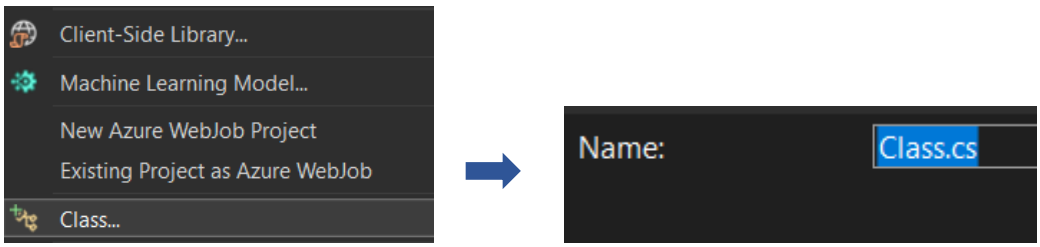
- ***Context Class***: Model class and SQL server cannot directly connect with each other for their connection we have to use core entity framework which we installed previously and we have to create another db context class.
- For this right click model click on Add & then new folder.




- Rename & in ***data*** folder create ***class***.

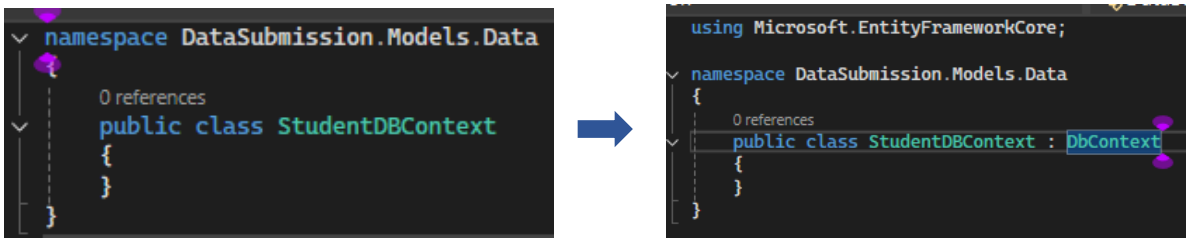


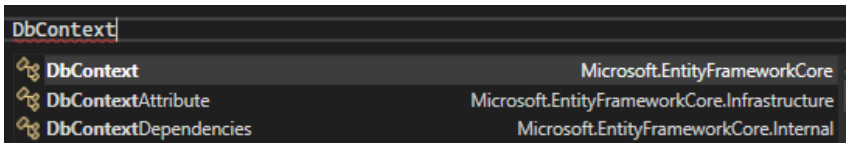

- Change class name from bottom bar:



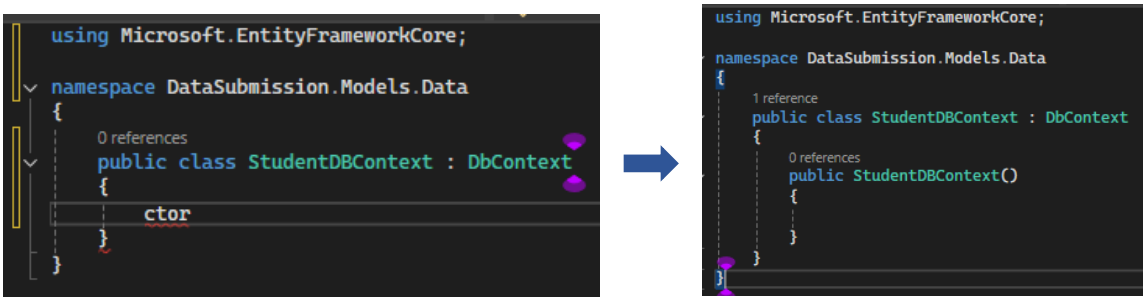
- Write Name like ***StudentDBContext.cs*** & ***Add***


- ***Inherit*** the class with actual already created ***DBContext*** class.

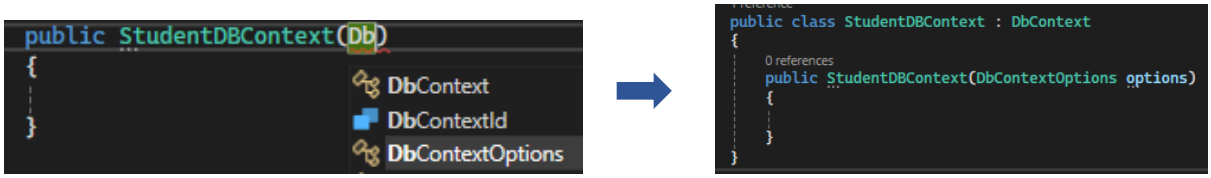


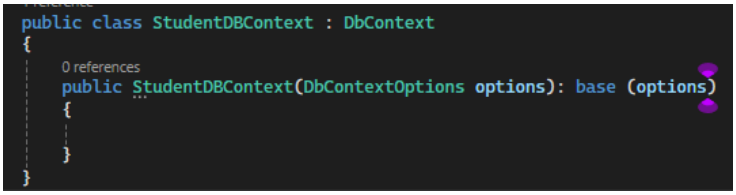

- Creating Constructor: type ***ctor*** and press ***tab***.




- Give a parameter ***option***.




- ***Inherit*** from base class Student ***DBContext***

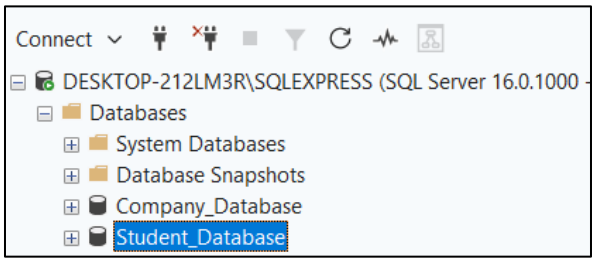



- ***Connect Model class column*** with context class for this type ***prop*** press Tab and type ***DbSet<Student> Student{get,set}.***

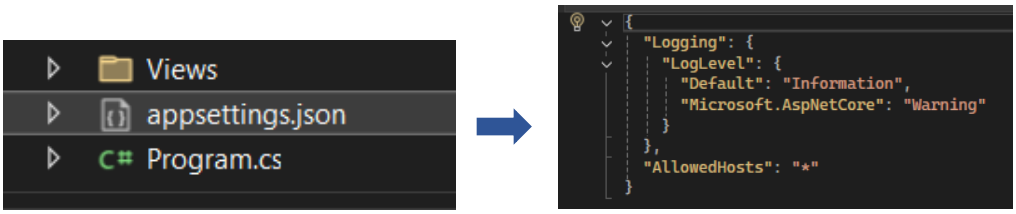
```
using Microsoft.EntityFrameworkCore;

namespace DataSubmission.Models.Data
{
    1 reference
    public class StudentDBContext : DbContext
    {
        0 references
        public StudentDBContext(DbContextOptions options): base (options)
        {

        }
        0 references
        public DbSet<Student> Student { get; set; }
    }
}
```

```
Step 4: Add Connection String in appsettings.json
- Add connection string with key DefaultConnection
- "ConnectionStrings": {
    "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True}
```

- ***Studend_Database created:***



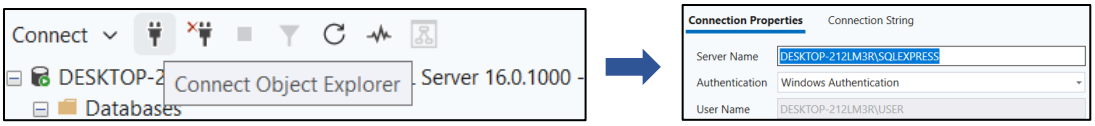

- ***Add connection string*** in ***appsetttings.json:***
- Now we want connection of SQL server & the project we created for it go to solution explorer second last option app settion.json.



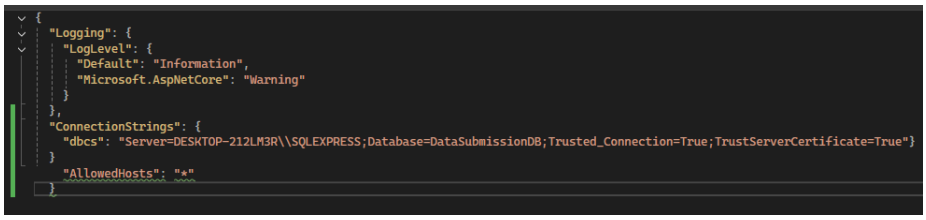

- "ConnectionStrings": { "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True} ***copy this path*** to the .json file & ***add your server name***.

```
Step 4: Add Connection String in appsettings.json
- Add connection string with key DefaultConnection
- "ConnectionStrings": {
    "DefaultConnection": "Server=your_server_name;Database=your_database_name;Trusted_Connection=True;TrustServerCertificate=True}
```

- For ***server name*** go to the SQL server ***copy server name*** and ***paste it in .jsoon*** file.



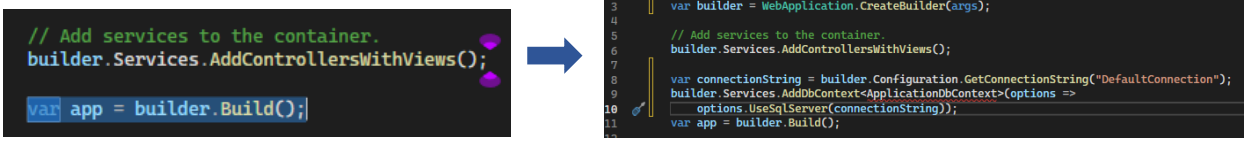

- Also ***add your database name*** like ***DatasubmissionDB*** now a database of this name with student table created automatically on SQL server.


```
Step 5: Register Connection String in Program.cs
- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
```

- ***When project run*** it direct comes in ***program.cs*** now we ***register connection string in program.cs***
- For it ***paste*** “var connectionString = builder.Configuration.GetConnectionString("DefaultConnection"); builder.Services.AddDbContext<ApplicationDbContext>(options => options.UseSqlServer(connectionString));” in program.cs file change the database name with the created database name.

```
Step 5: Register Connection String in Program.cs
- Read connection string from configuration.
- Register DbContext using AddDbContext and SQL Server provider.
- var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
```




- In program.cs file change the database name with the created database name.

```
var connectionString = builder.Configuration.GetConnec
builder.Services.AddDbContext<Stud>(options =>
    options.UseSqlServer(conne
var app = builder.Build();
```
Student
StudentDBContext

```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

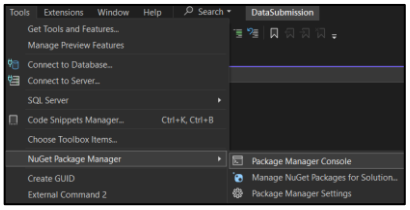
- Also ***change default connection*** with ***dbcs*** we named it in ***.json file***

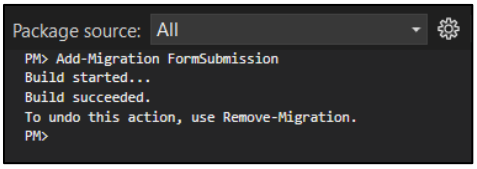
```
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

```
var connectionString = builder.Configuration.GetConnectionString("dbcs");
builder.Services.AddDbContext<StudentDBContext>(options =>
    options.UseSqlServer(connectionString));
var app = builder.Build();
```

```
Step 6: Migration & Update Database
- Use Package Manager Console or CLI:
  - Add-Migration InitialCreate
  - Update-Database
```

- ***Migration and update database***: we do it at the time database & table will created in SQL.
- For this go to ***Tools*** in ***nugget Package manager*** go to ***package manager console***. Type Add ***Migration command***:

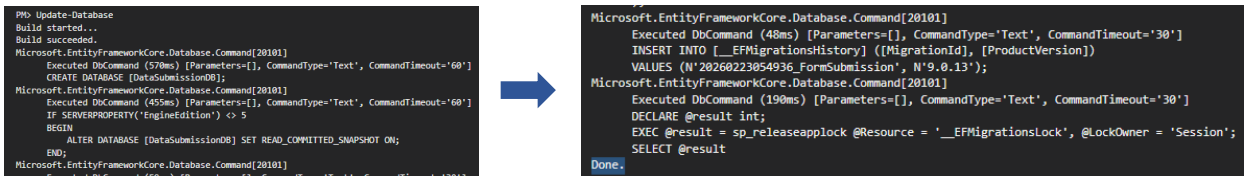


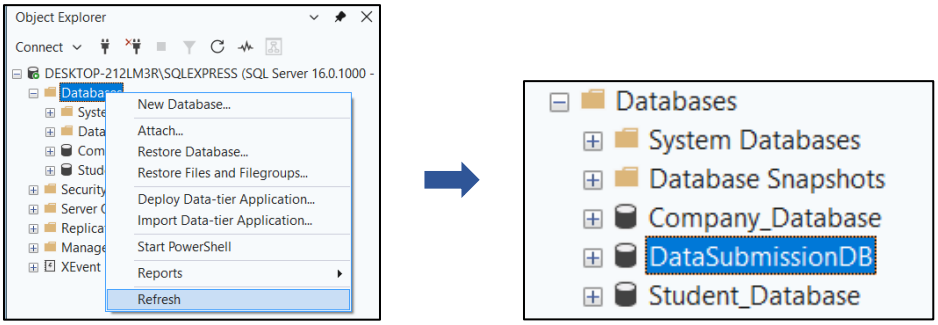

- ***Add Migration file***:Detail of file show (name, email, gender) & primary key etc.

```
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace DataSubmission.Migrations
{
    /// <inheritdoc />
    public partial class FormSubmission : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "Student",
                columns: table => new
                {
                    id = table.Column<int>(type: "int", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    StudentName = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    StudentEmail = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    StudentPassword = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    StudentGender = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    age = table.Column<int>(type: "int", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Student", x => x.id);
                });
        }
```

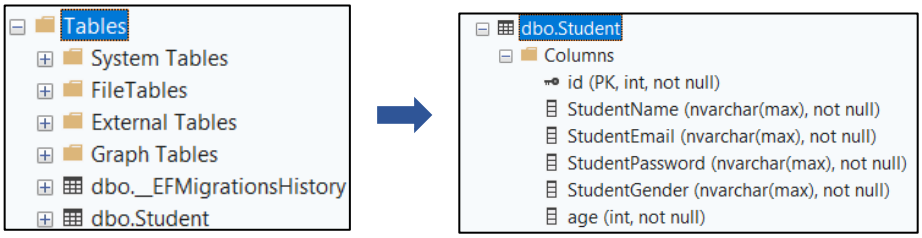
- ***Second command***: Update command



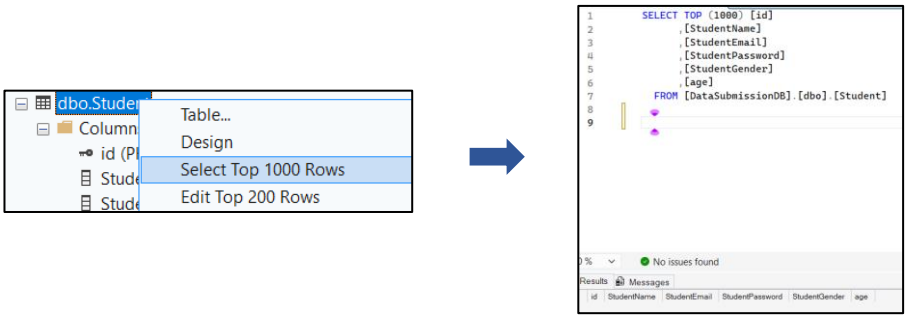

- Firstly ***Refresh*** then check whether the ***FormSubmission*** database with student table in SQL server is created or not.




- In ***Tables***, All the ***columns*** we created in Modal class show here.



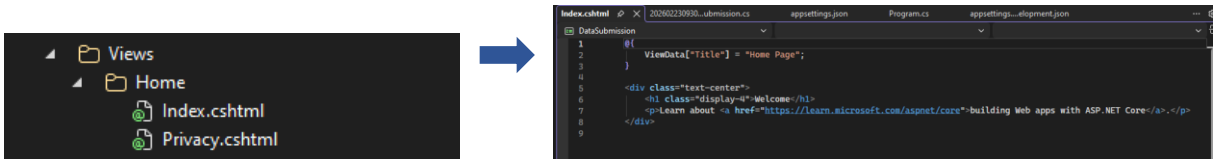

- Now check if there is values or not. For now no value show in it.



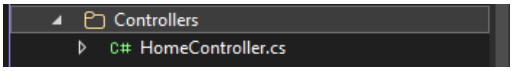

```
Step 7: Controller Injection and Create User Form Page (Create Template)
- Controller with constructor injection of DbContext.
- Scaffold Create Razor page for Student.
- Fields: Name, Email, Password, Gender, Age.
```

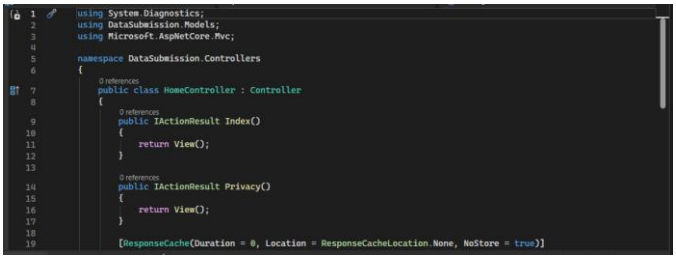
- Now this step, when the ***user fills form enter data*** such as ***name email gender*** etc & ***submit*** the form the data will show in above screen.

- Controller Injection & ***Create User Form*** Page (Create Template). Controller with constructor injection of ***DbContext***. Now go to ***Controller page*** & make it's ***connection*** with database.
- ***User interact with index page*** in views but this ***index page is not directly contact*** with database it is connected to Controller and controller is connected to database.

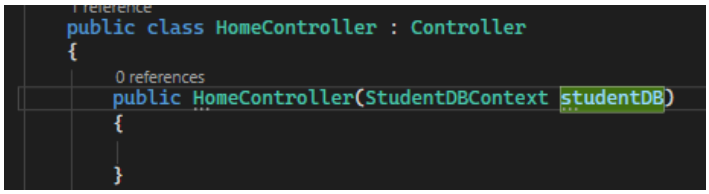



- Now go to ***home controller.cs*** page

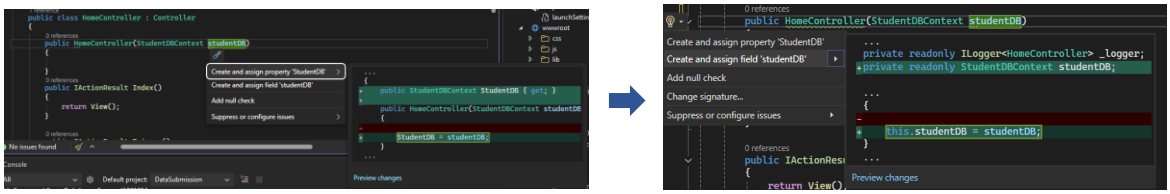


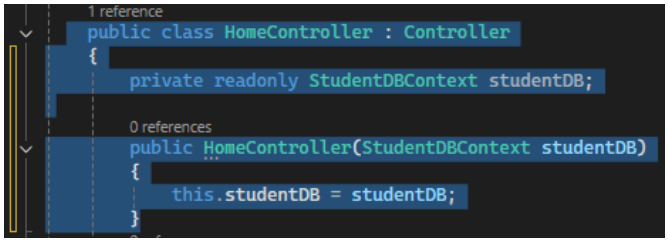
- Now ***make a constructor*** on IsAction Result ***Index().*** The constructor has same name of class & give it ***parameter of the DB context*** class and give a name to object ***studentDB***



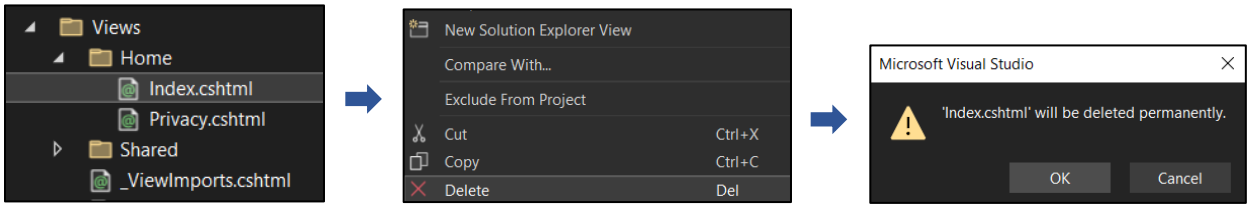
- Now point on ***studentDB*** press ***Ctrl+.*** , this screen appear. ***Go to second option*** Create and ***Assign field"studentDB***".



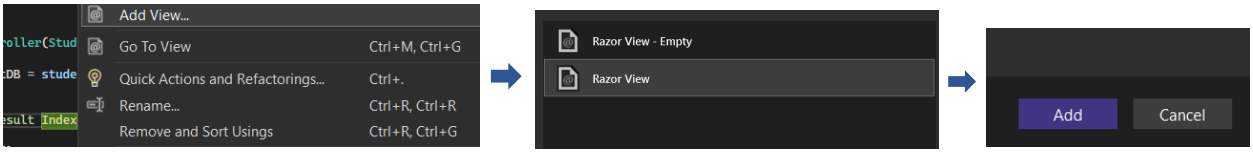
- this.studentDB = studentDB; now we created connection with database. This is called injection.



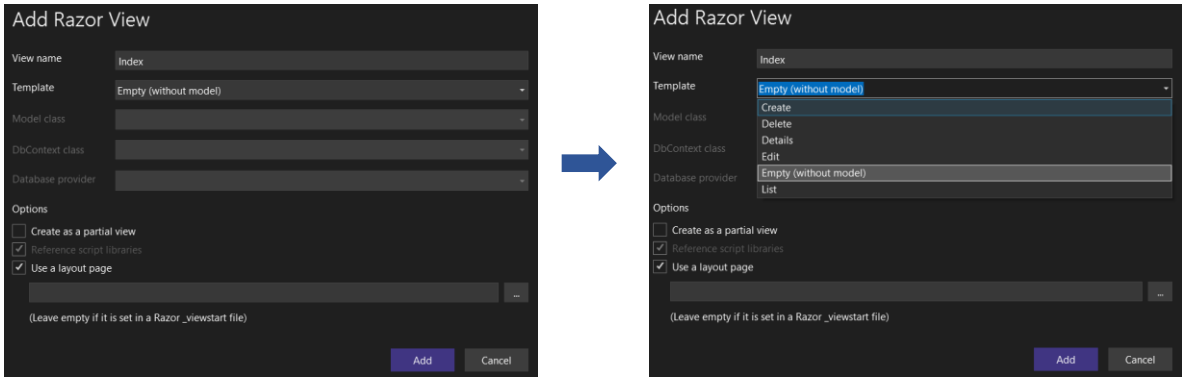

- Now we go to ***user view*** the ***index.cs html*** & ***delete*** it then ***OK***.




- Now we ***want to create new view named index***. For this ***right click Index*** and ***go to Add View***. Then ***go to Razor View***. Razor view is ***use when we want to insert c# code in html*** file the razor engine then covert the c# code in html. Now ***click on Add***.

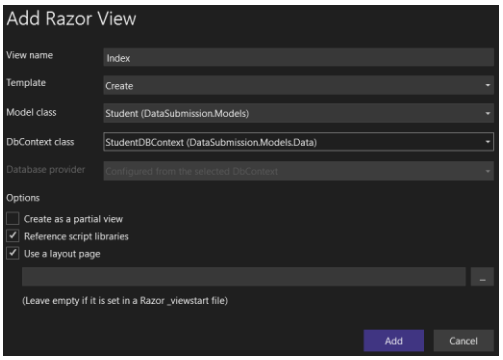


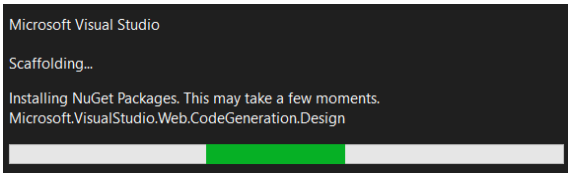

- Now this screen appear & ***change the template*** to ***Create*** due to it the table we created will automatically change to form.



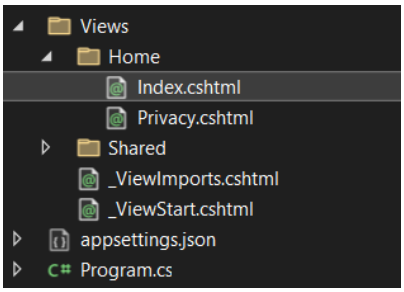

- Enter the ***Modal*** and ***DB class name***.

- Click on ***Add*** Process start, now view file is creating.



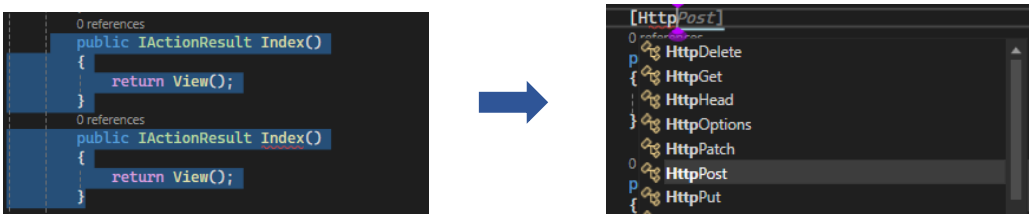

- ***Form creation successful***. <**form asp-action**="Index"> it means we want to run index action in controller.





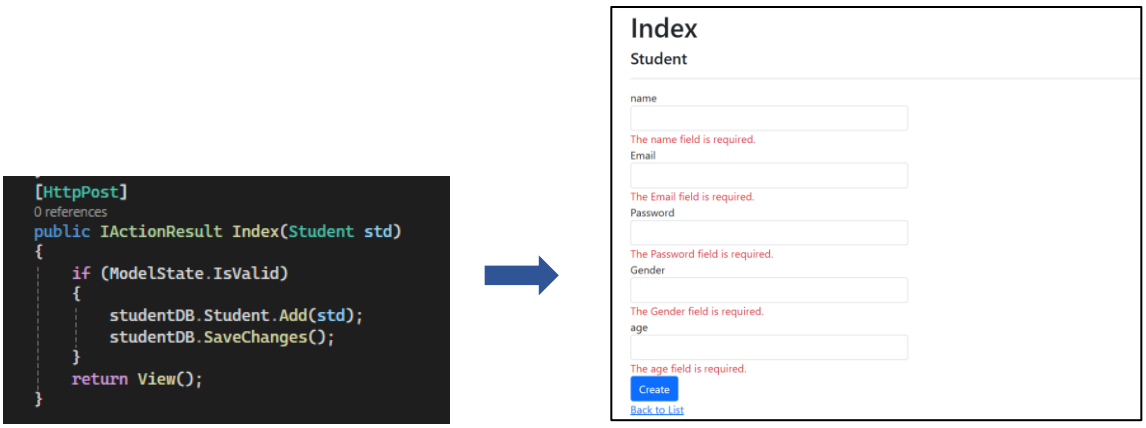
```
Step 8: HTTP POST Action to Submit Data
- [HttpPost] action accepting Student model.
- Validate ModelState.
- Save data using DbContext.
```

- ***HTTP POST Action*** to Submit Data(sending data from user to database)
- Now ***the http post action to submit data*** that when user enter data in above form it should submit. ***To tell the controller about post action*** go to ***controller file*** & write the ***constructor*** again & ***above it write*** [HTTP post]




- ***Give modal class*** & it's ***object as parameter***. Now ***when user enter data*** & click on ***Create it will store in std object***.


- Now ***Validate ModelState*** & ***Save data*** using ***DbContext***.
- For this give condition in ***Home Controller.cs***. HTTP post section that if all conditions in ***Model class is valid then save changes*** & ***add the values in std object*** in student DB to ***which Modal class is connected***. Now the ***data will save in SQL Server*** in database.
- Now if ***we don't enter all the fields*** then ***JavaScript validation*** will apply.



```
Step 9: Show Success Alert Message
- After successful submission, show alert: "Data successfully submitted" using TempData or JavaScript.
```

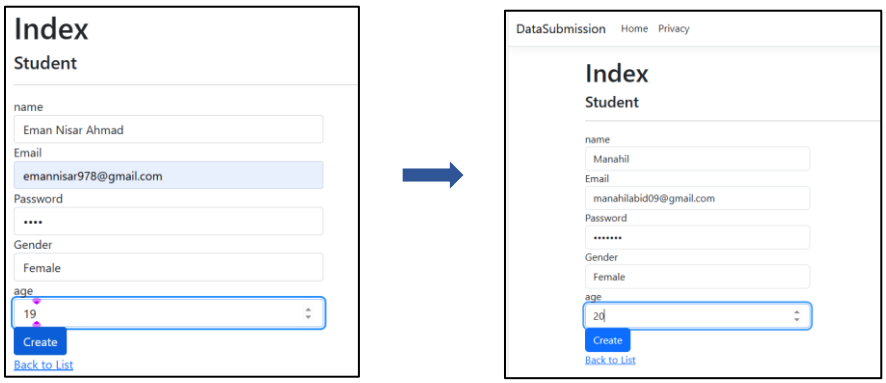
- Show ***Success Alert Message***, After ***successful submission***, show alert: "***Data successfully submitted***" use TempData or JavaScript. ***TempData used to show the c#code in html***.

```
[HttpPost]
0 references
public IActionResult Index(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Data Successfully submitted";
    }
    return View();
}
```

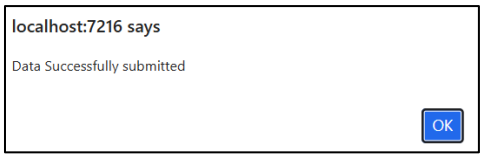
- Now to show it in html we use razor view syntax using@.
- After saving changes in program.cs file then this condition will apply.

```
@if (TempData["successmessage"] != null)
{
    <script>
        alert('TempData["successmessage"]');
    </script>
}
```

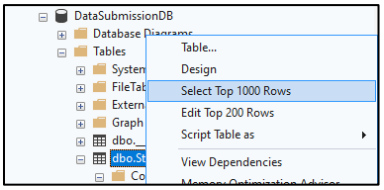
- ***Now run*** the form.




- For ***Data Successfully Submitted alert*** pop up click ***OK***.




- Now check the data should be store in SQL sever. Right click student table and click on Edit top 1000 rows.



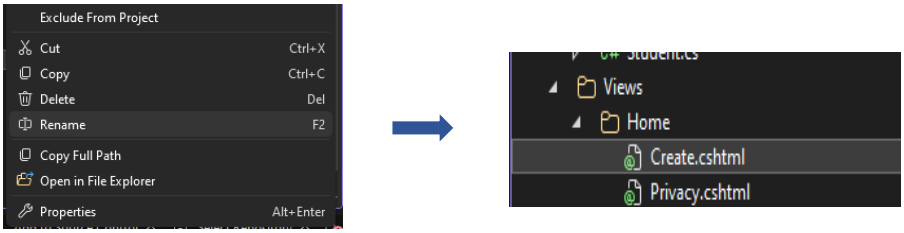
- Data appear successfully.


## Step 10: Design the User Form

- For this ***go to indexhtml.cs file*** & ***design the form as we want***.

## CRUD operations:

- ***Create***: previously we create a form now we want to form a create option for it so ***rename*** the ***index.cshtml file to create.cshtml***.



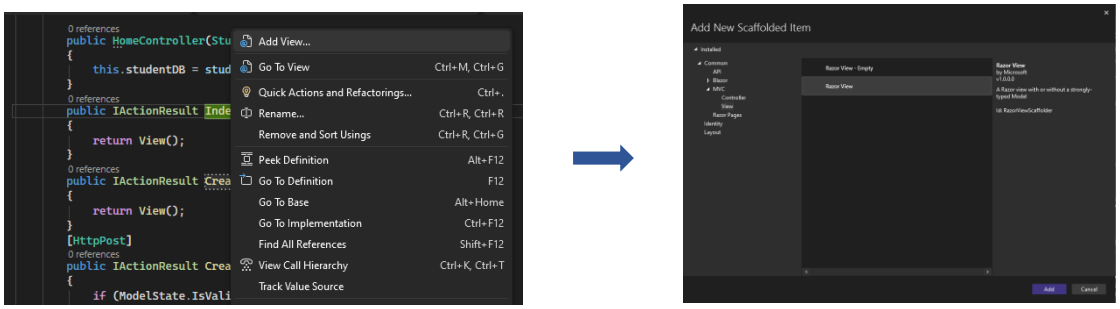

- ***Change the name in view*** & ***post method in Home Controller.cs*** also.

```
0 references
public IActionResult Create()
{
    return View();
}
[HttpPost]
0 references
public IActionResult Create(Student std)
{
```

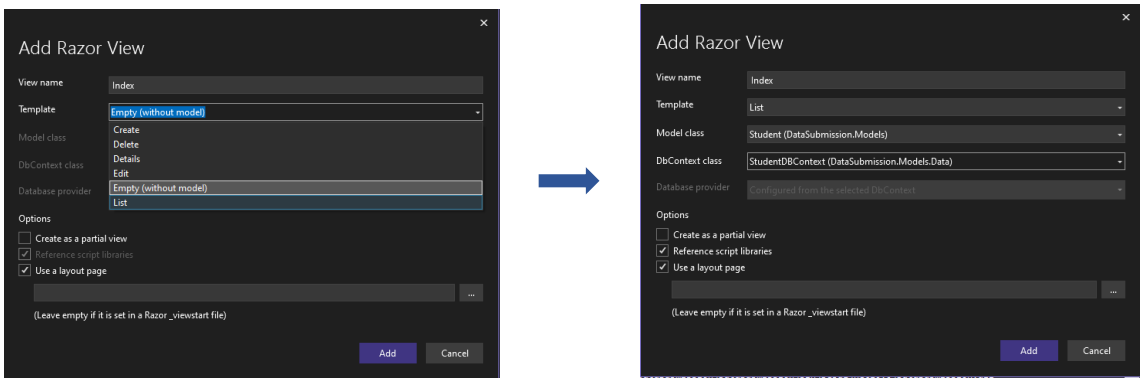
- Also ***change the action in Create.cshtml*** to Create which was Index previously & also ***define method*** which is post.

```
DataSubmission
@model DataSubmission.Models.Student

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>
<h4>Student</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="create"method="post">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="name" class="control-label"></label>
                <input asp-for="name" class="form-control" />
                <span asp-validation-for="name" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Email" class="control-label"></label>
                <input asp-for="Email" class="form-control" />
```

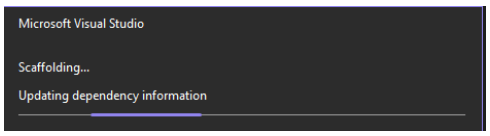
- ***Index Page***: Now we want that the data in SQL server will show on index page. For this purpose again go to the ***IsActionResult Index*** in ***Home Controller.cs*** right click it & click on ***Add view*** then go to ***Razor View*** & click on ***Add***.



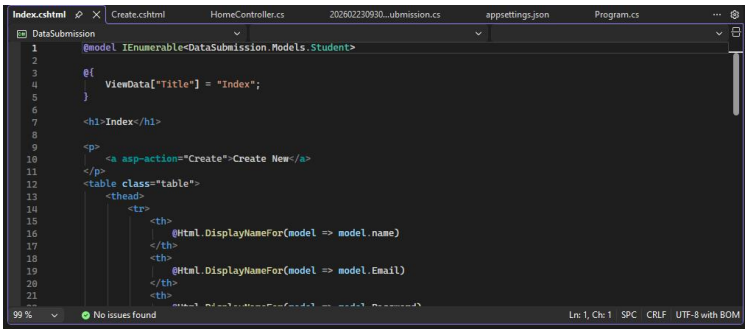

- ***Choose List from Template*** as we want to ***Read*** or ***view*** data in index page.
- ***Choose List select DB*** & ***Modal class*** & click on ***Add***.




- ***Creating*** Index page.




- Index page ***created***.

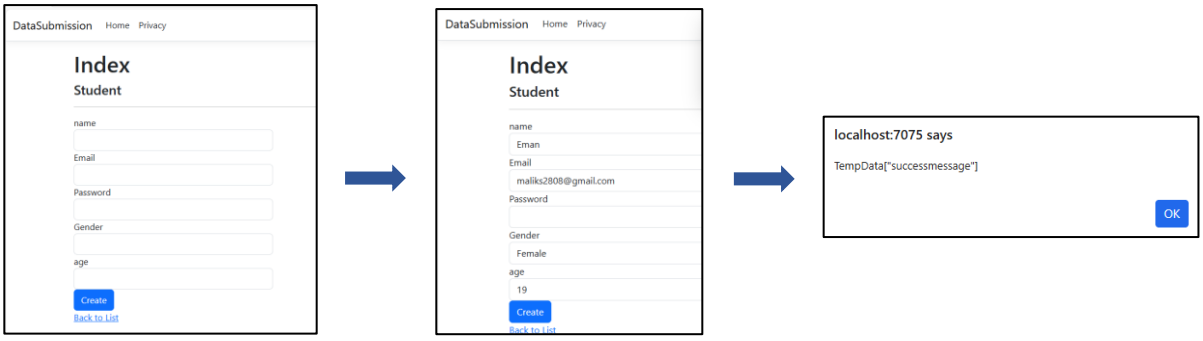


- ***In Home Controller*** we already created the index action now we want to show or view data we create a variable & in it ***we take studentDB instance of context class*** & ***show the Model class Student data***.

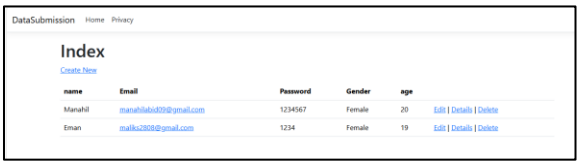
```
public IActionResult Index()
{
    var stData = studentDB.Student.ToList();
    return View(stData);
}
```

- Now ***Run*** it. we can see ***the Index page*** is showing first & in it there is ***the student data***.

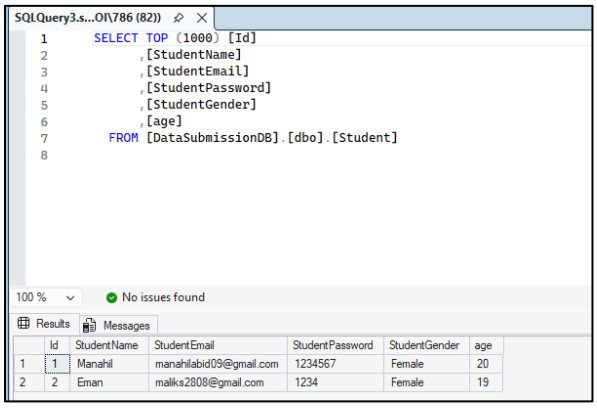

- We can also see ***create New***. ***Click*** on it form will show, Insert Data & ***OK for Pop up***.




- ***Go back to list data*** can be ***view in Index page***.




- Data is ***also visible in database***.



- ***Redirect on Home Page***: Now the next task is that when we enter data in form it redirects to Home page. For this ***go to Home Controller.cs*** & In it where we created message below it ***remove return view*** & ***add redirect***.
- ***Redirect ToAction("Index","Home")*** means redirect to Index page and controller will be Home.

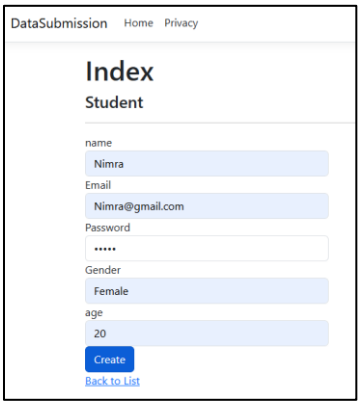
```
public IActionResult Create(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Data Successfully submitted";
    }
    return View();
}
```

```
    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Data Successfully submitted";
    }
    RedirectToAction("Index", "Home");
}
```

- In case ***if there is issue in redirecting*** then it return view.

```
26          [HttpPost]
            0 references
27          public IActionResult Create(Student std)
28          {
29              if (ModelState.IsValid)
30              {
31                  studentDB.Student.Add(std);
32                  studentDB.SaveChanges();
33                  //TempData["successmessage"] = "Data Successfully submitted";
34                  return RedirectToAction("Index", "Home");
35              }
36              return View(std);
37          }
```

- Now ***run*** & ***again insert data*** by clicking ***Create new it will redirect*** to Home page.

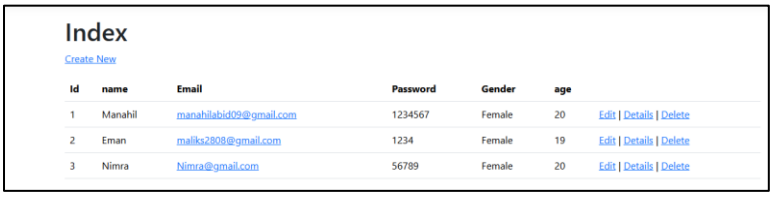




- ***Details***: If we want to show details for example id which is not visible here but in database id is also visible. For it ***go to index.cshtml*** & in ***table section add Display*** For loop for id.

```
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Id)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Email)
            </th>
```

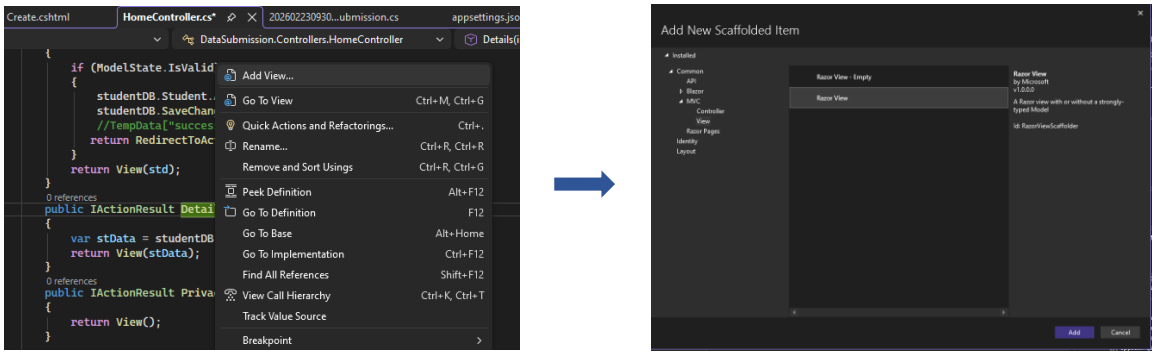
```
@foreach (var item in Model) {
        <tr>
            <td>
                @Html.DisplayFor(modelItem => item.Id)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.name)
            </td>
            <td>
                @Html.DisplayFor(modelItem => item.Email)
```



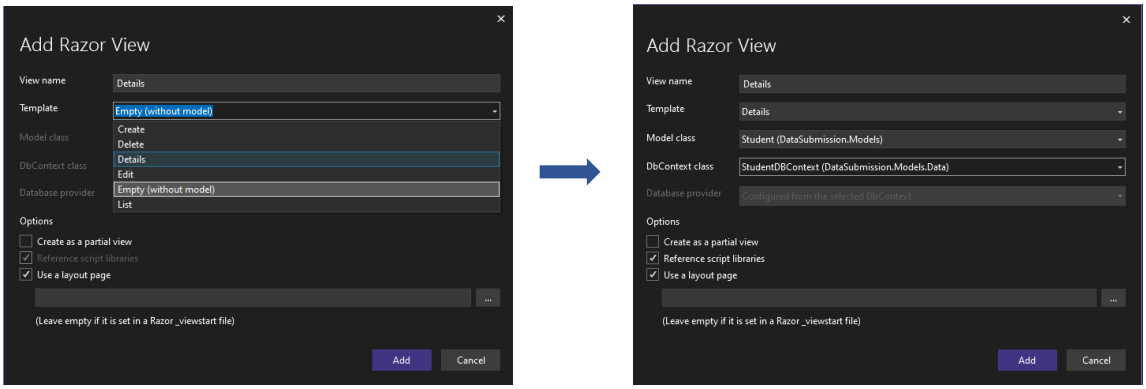
- Now to create detail page ***go to Home controller.cs*** create a Details action ***& give Id as parameter***.

```
public IActionResult Details(int Id)
{
    var stData = studentDB.Student.ToList();
    return View(stData);
}
```

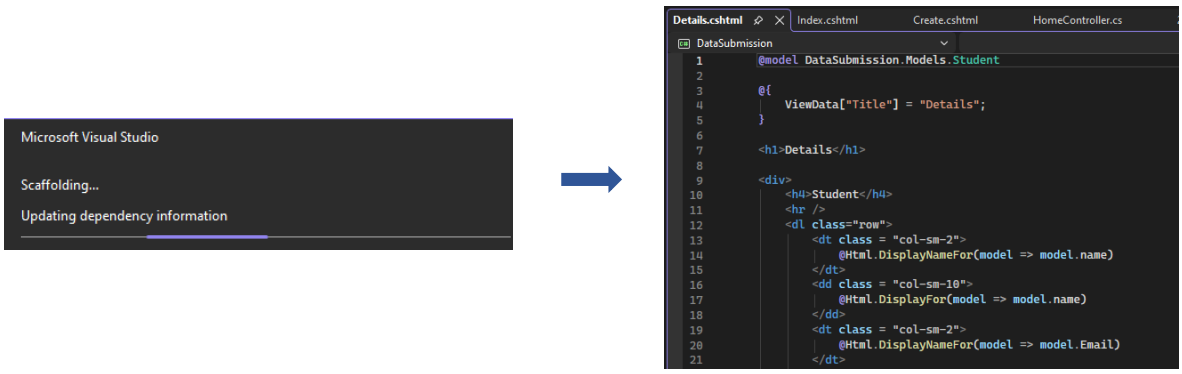
- Again create it's page ***right click the Details*** & click on ***Add view then Razor View*** & Click on ***Add***.




- ***Select Details template*** choose ***Model & DB classes*** and click on ***Add***.




- ***Creating*** page.



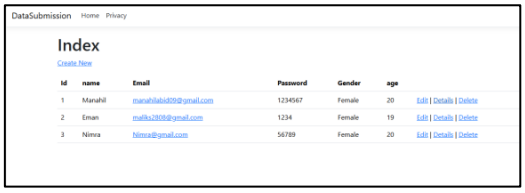
- ***Add Id*** list in it.

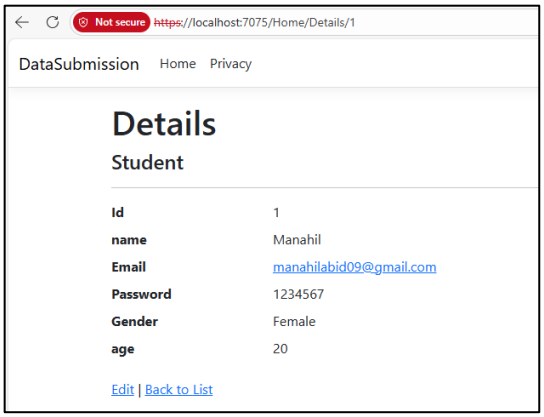
```
<div>
    <h4>Student</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Id)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Id)
        </dd>
```

- Now in ***Home Controller.cs*** add ***FirstOrDefault*** in variable ***stData***. It ***matches*** the ***parameters id with database Id*** if it found it ***show details*** otherwise if ***condition run*** which is "***if Id==null***" then ***Not Found***.

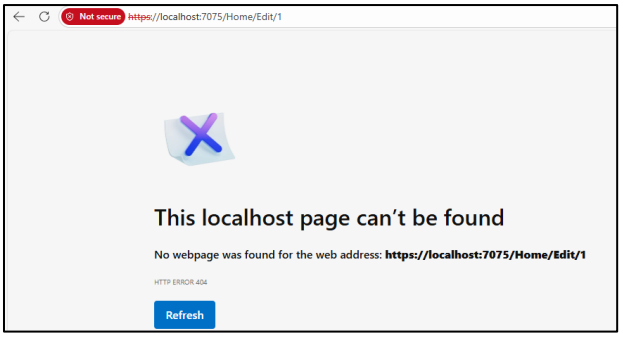
```
0 references
public IActionResult Details(int Id)
{
    var stData = studentDB.Student.FirstOrDefault(x=>x.Id==Id);
    if(stData == null)
    {
        NotFound();
    }
    return View(stData);
}
```

- Click on Details now the Details page is visible and it is taking Id as parameter.



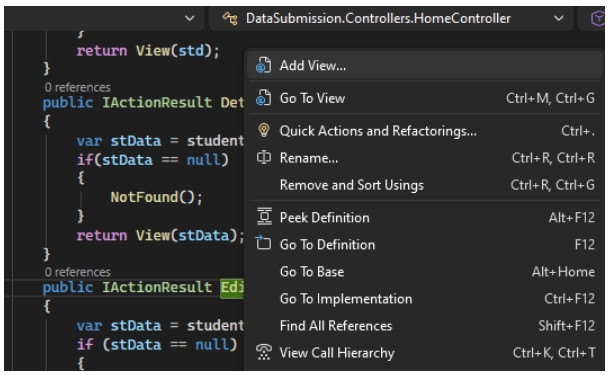

- ***Edit***: For now edit page ***cannot found*** as it is ***not created yet***.

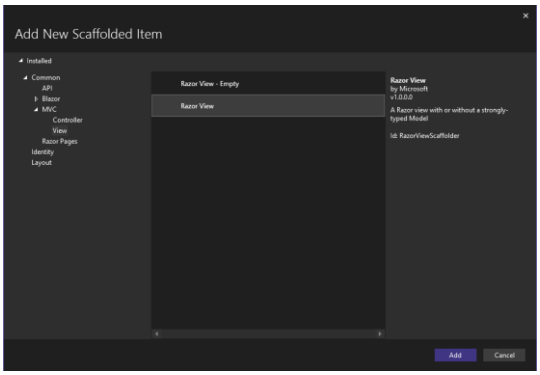



- For Edit page ***go to Home Controller.cs*** create Edit action & ***give Id as parameter*** now in edit one it ***finds the values*** in database using Id. If it ***finds data then return*** it otherwise show ***not found***.

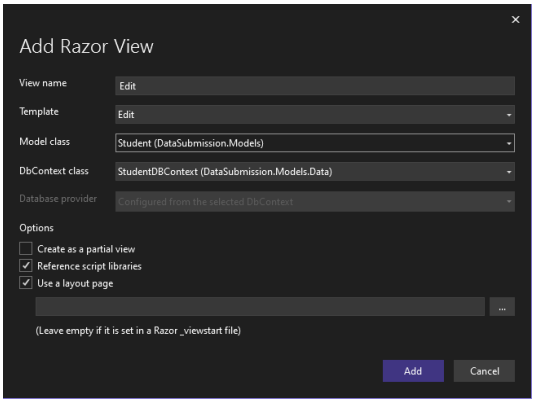
```
0 references
public IActionResult Edit(int Id)
{
    var stData = studentDB.Student.Find(Id);
    if (stData == null)
    {
        NotFound();
    }
    return View(stData);
```

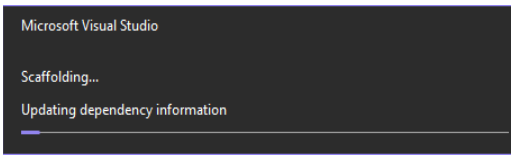
- Again create Edit page ***right click the Edit*** & click on ***Add view then Razor View*** & Click on ***Add***.

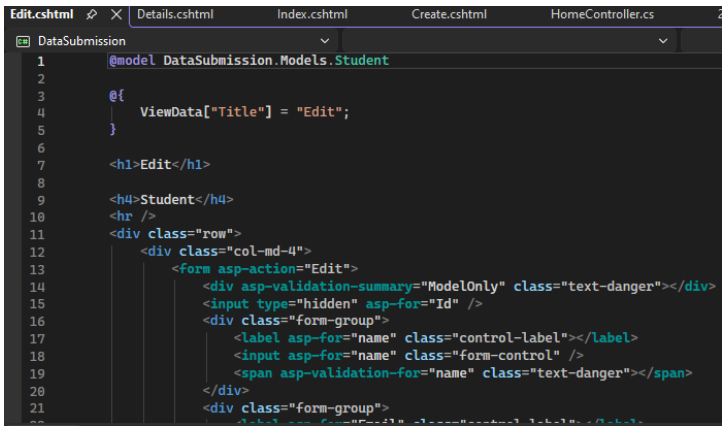


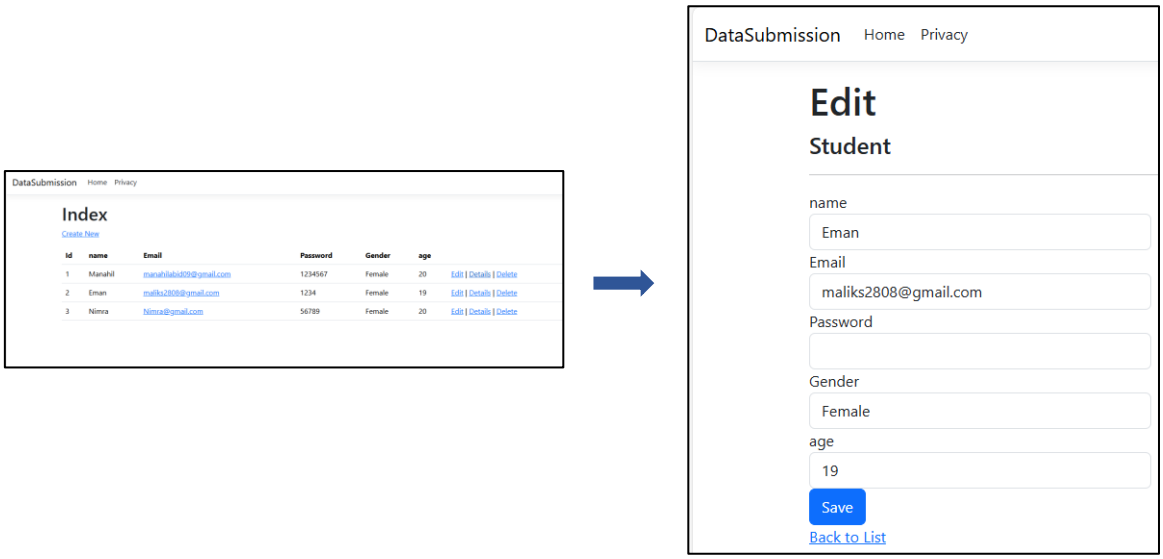

- Select ***Edit template*** choose ***Model and DB classes*** and click on ***Add***. Page Creating in process & ***created.***



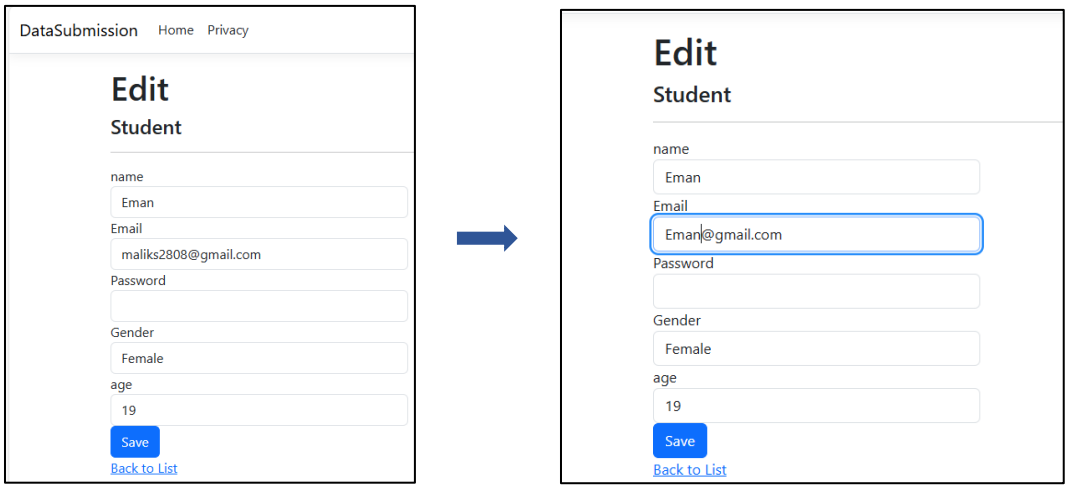
- Click on ***Edit***. Now ***Edit page is visible***.



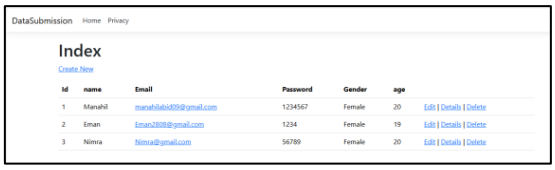

- Next step is ***the editing process*** that if we ***edit anything*** in this edit page it will also save in database. For this again ***go to Home Controller.cs*** we create ***Http Post action*** of Edit in which ***we use update*** that if the data is ***Valid*** then ***update it with previous one***.

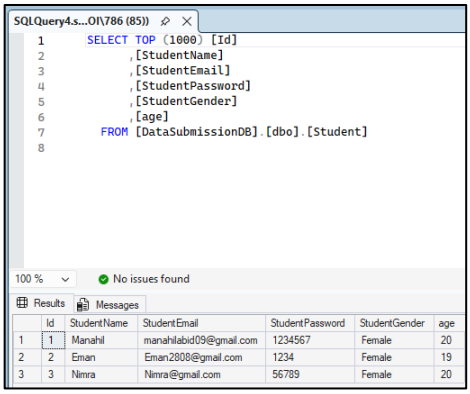
```
[HttpPost]
0 references
public IActionResult Edit(Student std)
{
    if (ModelState.IsValid)
    {
        studentDB.Student.Update(std);
        studentDB.SaveChanges();
        //TempData["successmessage"] = "Data Successfully submitted";
        return RedirectToAction("Index", "Home");
    }
    return View(std);
}
```

- Now we ***update the email*** of the student & click on ***save it will redirect to home page*** & ***changes will save*** & ***visible in both database*** & ***Index page***.




- Now we can see ***updated Email***. ***Email is updated in database*** also.

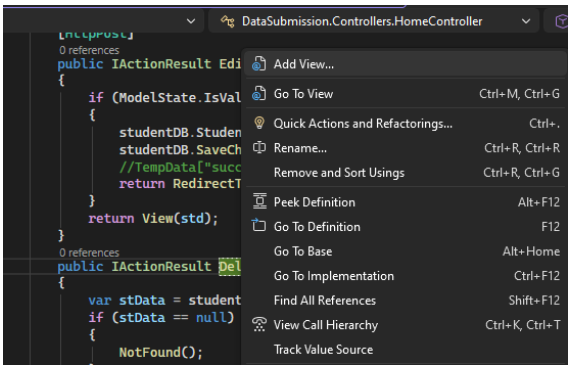


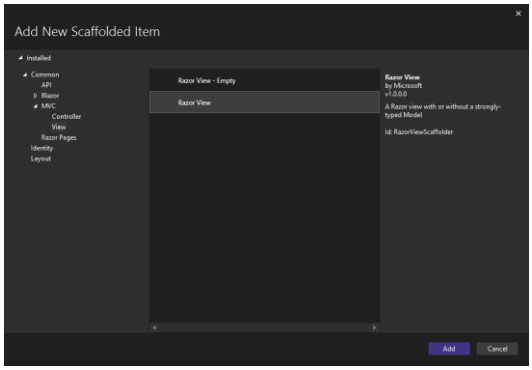




- ***Delete***: If we want to delete any student data. For his again create an action in Home Controller.cs.

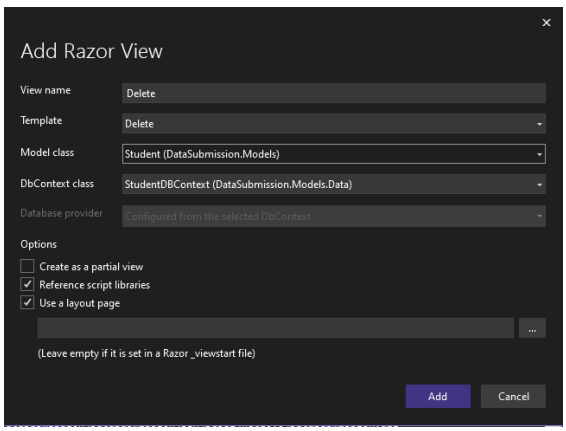
```
public IActionResult Delete(int Id)
{
    var stData = studentDB.Student.FirstOrDefault(x => x.Id == Id);
    if (stData == null)
    {
        NotFound();
    }
    return View(stData);
```

- Again create Delete page **right click the Delete** & ***click on Add view*** then ***Razor View*** & Click on ***Add***.

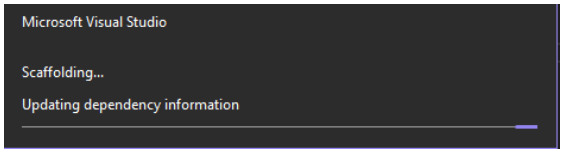


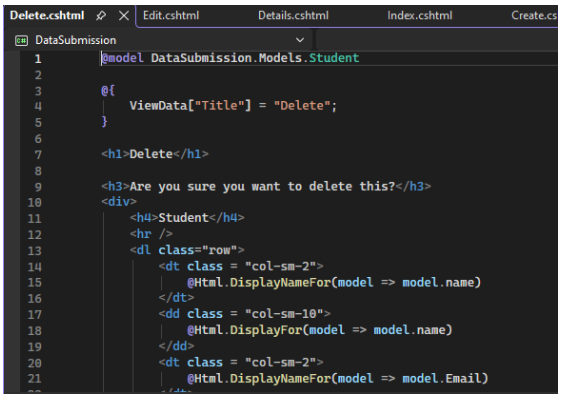




- Select ***Delete template*** choose ***Model and DB classes*** & click on ***Add***.

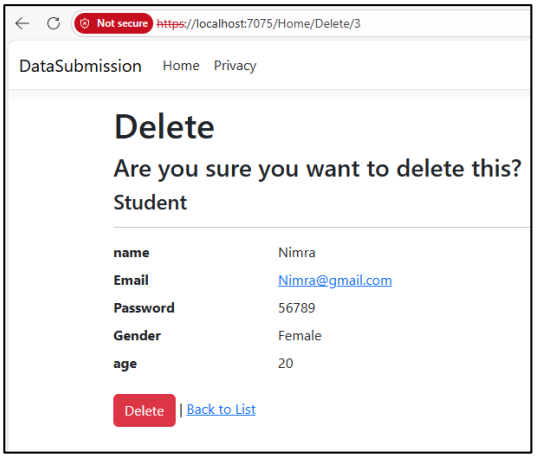


- ***Creating page*** in process then ***created***.

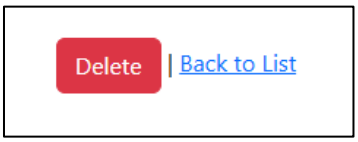


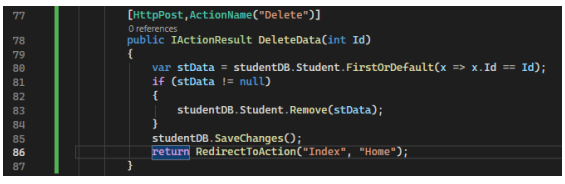




- Now ***we want that first the data show on page*** then ***Delete***. For this ***go to Home Controller.cs*** add ***FirstOrDefault*** in variable ***stData***. It ***matches*** the ***parameters id with database Id*** if it ***found it show details*** before deletion otherwise if condition run which is "***if Id==null***" then ***Not Found***. Click on ***Delete.*** It will take ***id as parameter***.

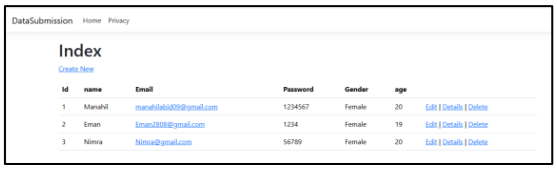



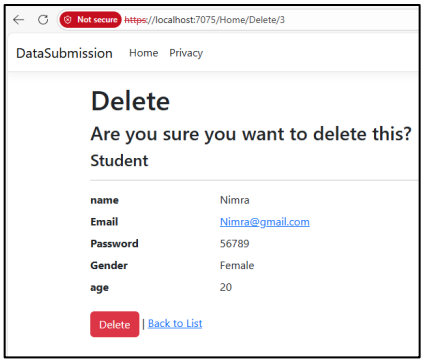

- Now working on the ***Delete button***. For this run ***Http post function*** for it in ***Home Controller.cs.*** Give it Action Name "***Delete***" so that ***post action*** will run ***after delete action***. In Post action it will ***check if the data is not Null*** then ***remove*** the Model class student data & ***save changes***.

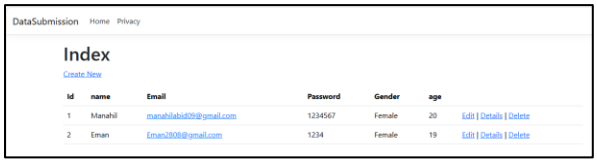


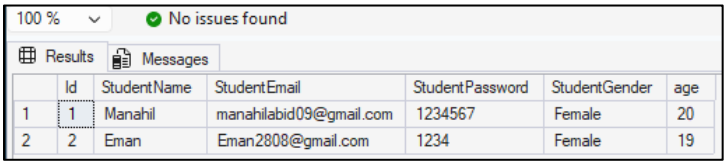

- Now ***click on delete*** now it will ***redirect to Index page*** & now ***student 3 data is deleted***.



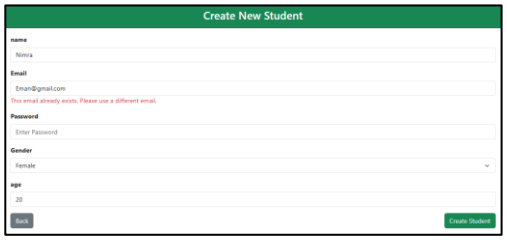




- ***Data deleted*** in ***database*** also.



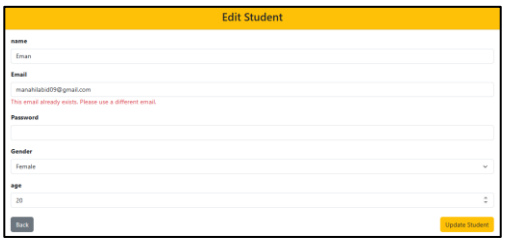

- ***Email Validation***: Make sure no student enter duplicate email.
- For this ***update the Create*** & ***Edit post actions*** in ***Home controller.cs*** & ***add email validation in it***. Create ***post action update***:

```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public IActionResult Create(Student std)
{
    // Check if the email already exists
    if (studentDB.Student.Any(s => s.Email == std.Email))
    {
        // Add error to ModelState
        ModelState.AddModelError("Email", "This email already exists. Please use a different email.");
    }

    if (ModelState.IsValid)
    {
        studentDB.Student.Add(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Student added successfully!";
        return RedirectToAction("Index");
    }
```




- ***Edit*** post action update:

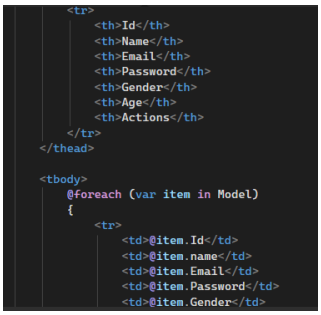
```
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public IActionResult Edit(Student std)
{
    // Check if email exists for another student
    if (studentDB.Student.Any(s => s.Email == std.Email && s.Id != std.Id))
    {
        ModelState.AddModelError("Email", " This email already exists. Please use a different email.");
    }

    if (ModelState.IsValid)
    {
        studentDB.Student.Update(std);
        studentDB.SaveChanges();
        TempData["successmessage"] = "Student updated successfully!";
        return RedirectToAction("Index");
    }
```

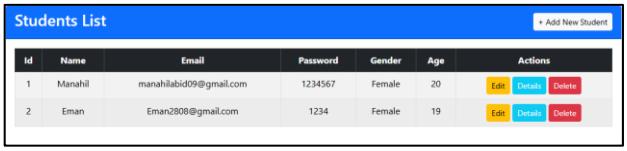



- ***Designing and alert***: For this go to index, cshtml file. Design the table tag.

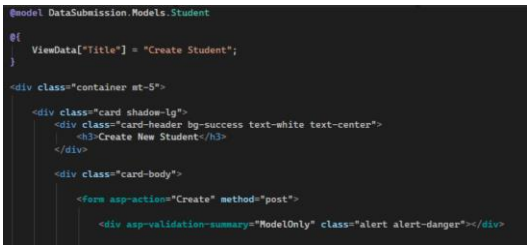
```
<table class="table table-dark table-active table-bordered table-hover">
    <thead>
        <tr>
```

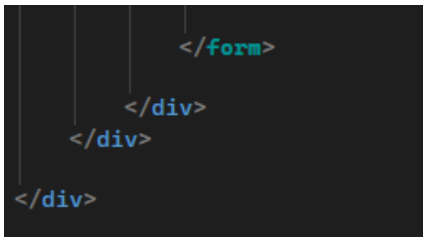

- Index ***page designing***:

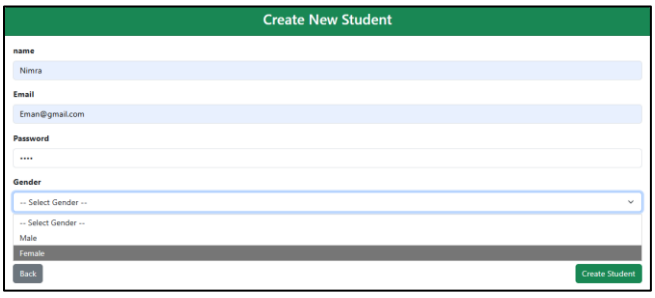




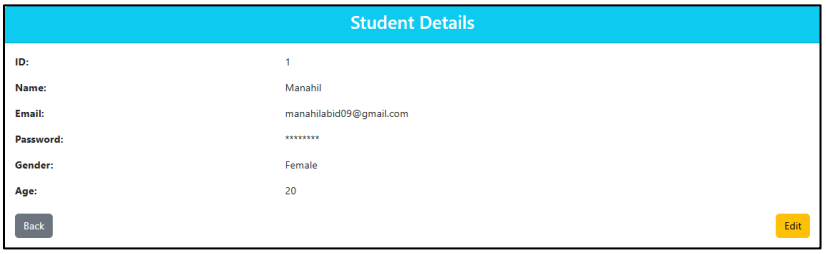

- ***Create*** page:



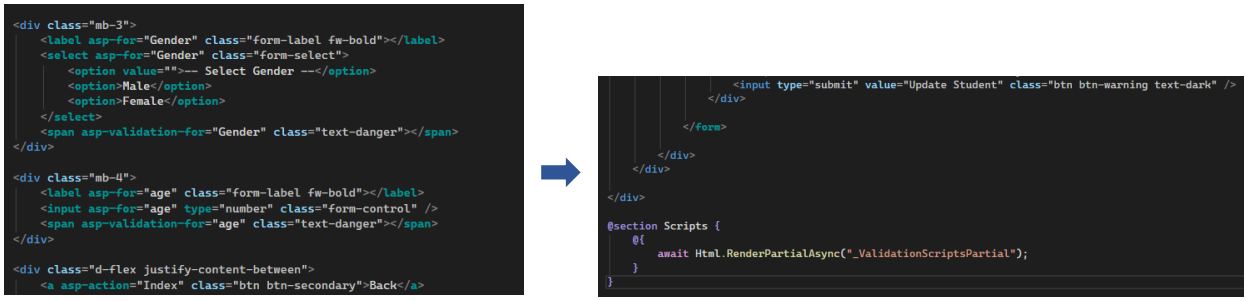


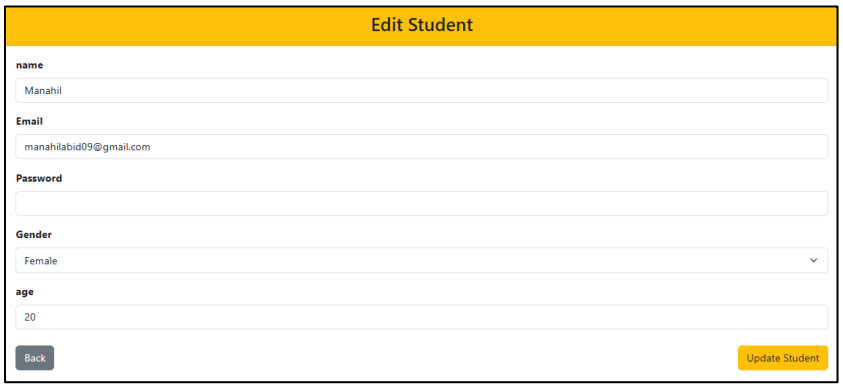


- ***Details*** page:

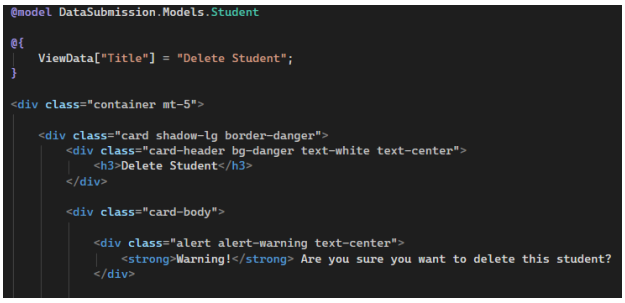



- ***Edit*** page:






- ***Delete*** page:


@model DataSubmission.Models.Student
@{
ViewData["Title"] = "Delete Student";
}

<h3>Delete Student</h3>

Warning! Are you sure you want to delete this student?

Name:
@Model.name

Email:
@Model.Email

Password:
********

Gender:
@Model.Gender

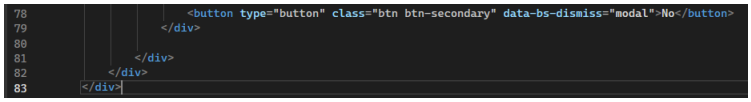
Age:
@Model.age

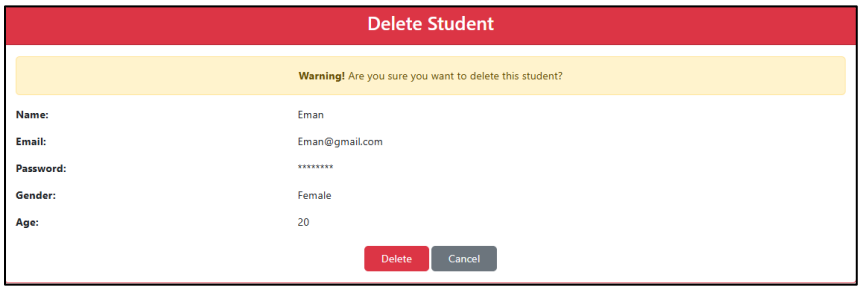
<button type="button" class="btn btn-danger px-4" data-bs-toggle="modal" data-bs-target="#confirmDeleteModal">
Delete
</button>
<a asp-action="Index" class="btn btn-secondary px-4">Cancel</a>

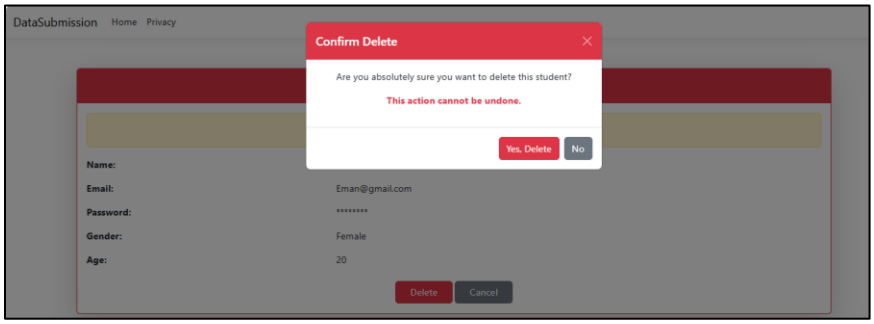
<h5 class="modal-title">Confirm Delete</h5>
<button type="button" class="btn-close btn-close-white" data-bs-dismiss="modal"></button>

Are you absolutely sure you want to delete this student?
This action cannot be undone.

<form asp-action="Delete" method="post">
<input type="hidden" asp-for="Id" />
<button type="submit" class="btn btn-danger">Yes, Delete</button>
</form>


78 <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">No</button>
79 
80
81 
82 
83 


Delete Student
Warning! Are you sure you want to delete this student?
Name: Eman
Email: Eman@gmail.com
Password: ********
Gender: Female
Age: 20
Delete
Cancel


DataSubmission
Home
Privacy
Confirm Delete
Are you absolutely sure you want to delete this student?
This action cannot be undone.
Yes, Delete
No
Name:
Email: Eman@gmail.com
Password: ********
Gender: Female
Age: 20
Delete
Cancel

# ------- THE END! -------